

Speaker Notes

Contents

- [1 1. Finishing I/O, and advanced I/O](#)
- [2 2. Recap by prediction](#)
- [3 3. Predict: draw last week's stack](#)
- [4 4. It is not one buffer — it is a stack](#)
- [5 5. Explain: what dies with the process, what dies with the machine](#)
- [6 6. Predict now, and keep it for an hour](#)
- [7 7. What dies with the process](#)
- [8 8. Where Lecture 2 stopped](#)
- [9 9. Three lines that print nothing](#)
- [10 10. Why the file is empty](#)
- [11 11. What this demonstration does not prove](#)
- [12 12. Predict: what will `ltrace` show that `strace` did not?](#)
- [13 13. `ltrace` : the same idea, one layer up](#)
- [14 14. Which levels can you actually watch?](#)
- [15 15. The same write, at three levels](#)
- [16 16. The same write, at three levels](#)
- [17 17. Meet the program: `iodemo.c`](#)
- [18 18. Three calls in the source. One in the trace.](#)
- [19 19. `fwrite` takes a size and a count](#)
- [20 20. Two operating systems, side by side](#)
- [21 21. Process Monitor: what the kernel did](#)
- [22 22. NtTrace: a tracer for the Windows Native API](#)
- [23 23. Three Windows words you will need](#)
- [24 24. Predict: the same program, on Windows](#)
- [25 25. A real Windows trace](#)
- [26 26. A handle is not a file descriptor](#)
- [27 27. Two error namespaces in one return value](#)
- [28 28. The stable boundary sits at a different level](#)
- [29 29. Where the boundary sits is contested history](#)
- [30 30. Linux is the outlier, not Windows](#)
- [31 31. One program, two kernels](#)
- [32 32. What we have not looked at](#)
- [33 33. `write\(\)` returned success. The bytes are not on a disk.](#)

34 34. Buffering moves your errors

35 35. Predict: all your writes succeeded — can the close still fail?

36 36. Why a close can fail when every write succeeded

37 37. Observe: close it twice, and watch what the second close hits

38 38. Never retry a failed close

39 39. Networking and IPC are this same mechanism

40 40. Honest limits

41 41. Honest limits, continued

42 42. Lecture 2 ends here. The question does not.

43 43. So when is it safe?

44 44. The kernel's own count of what it has not written

45 45. Observe: the program finished at zero seconds

46 46. Five runs, three sizes

47 47. Five seconds in memory that belongs to nobody

48 48. Which rule fired, and how we know

49 49. Visibility is not durability

50 50. Predict: what does one `fsync` per record cost?

51 51. Observe: sixty times, for the same hundred writes

52 52. What `fsync()` promises, and what it does not

53 53. Windows: the same shape, and now measured

54 54. The two kernels write in opposite orders

55 55. Predict: your `fsync()` failed — what does the retry return?

56 56. Observe: the retry returns success, over data that was never written

57 57. Explain: the retry does not retry

58 58. What the engines did instead

59 59. Windows here is not "the same", and not "safe"

60 60. Sync the directory too

61 61. Write-then-rename, and why it needs a sync

62 62. The respectable opposite view

63 63. Log first — and what if the log write fails?

64 64. So when is it safe?

65 65. Lecture 4 starts here

66 66. What a pipe is

67 67. Blocking, non-blocking, and letting the kernel wait for you

68 68. Predict: a hundred pipes, and nobody writing

69 69. Where the question comes from

70 70. A thread each, and why that stopped working

71 71. Observe: two seconds, zero bytes, counted

72 72. What those counts do and do not say

73 73. `epoll` is three calls

74 74. Level-triggered and edge-triggered

75 75. Three ways to wait, and we watch all three

76 76. Readiness is permission to try, not a promise

77 77. The real difference is who owns the buffer

78 78. It is not a Linux/Windows split

79 79. The trap this demonstration sets

80 80. Four Windows words before the Windows pair

81 81. Predict: the same hundred pipes, on Windows

82 82. Observe: Windows, traced, both ways

83 83. Which numbers you may compare, and which you may not

84 84. What a ring is

85 85. Where both platforms ended up

86 86. Same mechanism, different clothes

87 87. Predict: which of the four calls changes?

88 88. Observe: the file run and the pipe run

89 89. The descriptor is the abstraction, not the file

90 90. Predict: the same program, writing to your terminal

91 91. Observe: three writes, not one

92 92. Explain: the destination changed the trace, at the C library, not the kernel

93 93. Predict: the network leg

94 94. Observe: both netcats, traced

95 95. Then what are `send` and `recv` for?

96 96. The same interface is not the same contract

97 97. Predict: the same three destinations, on Windows

98 98. Observe: the API changes the call, the object does not

99 99. Windows: a `SOCKET` is not a `HANDLE`

100 100. Bytes you never touch

101 101. Predict: three buffers, one message

102 102. Observe: three lines become one

103 103. Gather buys crossings, not copies

104 104. Two promises, and only one of them is unconditional

105 105. Predict: moving a file you never look at

106 106. Observe: half the calls, and no trip through your program

107 107. What the name does not cover

108 108. Zero-copy moves bytes faster. It does not make them safer.

109 109. Windows, and a silent fallback

110 110. Homework, and the next question

111 111. Assignment 1 is not set today

112 112. Voluntary homework, and three rules for it

113 113. Summary – what follows from this

114 114. What Lecture 4 answers

115 115. References

Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.

Slides: 115 · **Duration:** 90 min · **Venue:** Faculty of Informatics, Masaryk University (FI MUNI)

11. Finishing I/O, and advanced I/O

On screen:

PB158 · Lecture 3

Your program never talks to the disk, the network, or another program. It talks to the layer next to it. Every layer says done about something different [V50] [inference].

*Ondřej (Ondra) Krajíček me@ondrejkrasicek.com · www.linkedin.com/in/OndrejKrajicek
Built at render time*

Welcome back. Last week we traced one write call down the stack. We stopped at slide thirty-two, and we stopped on a question.

Today has two halves. The first half finishes Lecture 2. Nothing is dropped and nothing is rewritten. The second half answers the question Lecture 2 left open.

The sentence on the screen is the whole lecture. Your program never talks to the disk. It talks to the layer next to it. That layer returns, and the return value says done. That word names a different place at every layer.

One hedge on my own sentence, because I hold you to this standard. My note says almost every fast interface behaves this way, not every one [V50]. I have written the sharper version on the cover and I mark it as mine [inference].

Keep one question in your head all hour. Whose done did I measure? A layer returns when it has accepted responsibility for the work, not when the work has happened [V50]. You will use it three times today, and I expect you to use it long after this course [speculative].

One practical note before we start. Every demonstration today is a real run on this machine. I have no screenshots. If something breaks, you watch it break.

2 2. Recap by prediction

On screen:

Part 1

Three minutes, on paper, before I show you anything.

Three minutes. Paper out, laptops closed.

I am not going to put last week's diagram back on the screen and read it to you. You would nod, and you would not remember it in an hour.

Instead you draw it. Then I show you mine, and you find out what you actually kept.

3 3. Predict: draw last week's stack

On screen:

Part 1 · Recap

Three minutes. Nothing on the screen until you have written something down.

1

Draw it

The four levels between your program and the medium. Name each one.

2

Mark it

After fflush() returns, which level are the bytes in?

3

Answer it

The machine loses power right now. What is lost?

Write all three. Nothing is handed in. I will ask several of you to read yours out [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

Three tasks. Three minutes.

First, draw the stack. Four levels between your program and the medium, each one named. If you can only remember three, draw three. Leave a gap for the fourth.

Second, mark where the bytes are after `fflush` returns. One level. Circle it.

Third, and this is the one that matters today. The power goes out this second. What have you lost?

Nothing is handed in. I will ask two or three of you to read out what you wrote. I want the wrong answers as much as the right ones. A wrong prediction you wrote down is worth more to you than a right answer you copied [S51].

Go. Three minutes.

I am watching the clock, not your paper.

Time. Hands down. Pens down. Look up.

The next slide is Lecture 2's own diagram, carried across unchanged. Put it beside your drawing and find out what you kept.

4 4. It is not one buffer — it is a stack

On screen:

Part 1 · Recap

[code]

Levels 1 and 2 are alternative routes into level 3 [V6]. Each arrow is one call [S17] [S3] [S13] [V37] [V33].

[S17] `io.IOBase.flush()` — Python documentation. [S3] `fflush(3)` — Linux man-pages.

[S13] `fsync` — The Open Group.

Four levels. Read them from the top [inference].

Level one is your program's own buffer, owned by the language runtime. Level two is the C library's buffer. Level three is the operating system's own memory for file data. Level four is the cache inside the storage device. Then, at the bottom, the medium.

Notice that levels one and two sit side by side. They are alternative routes, not a pile. A Python program uses the buffer at level one and calls the kernel directly. A C program uses the buffer at level two. I come back to this in four slides, because it is the detail people get wrong [V6].

Now read the arrows. Each one is a different call, and each one moves your bytes down exactly one step. No single call takes data from the top to the bottom.

Look at the bottom arrow. I come back to it, and I am not going to skip it.

5 5. Explain: what dies with the process, what dies with the machine

On screen:

Part 1 · Recap

Two different failures. Part 2's demonstration tests one. Your recap question asked about the other.

The process dies

Levels 1 and 2 go with it. Anything already handed to the kernel survives, because the kernel writes it back on its own [V14].

The machine loses power

Levels 1, 2 and 3 are gone. Level 4 goes too, unless the device has power-loss protection [inference].

My own note is blunt: "the honest classroom demo is a power-cut, not a `kill -9`" [V14]. Part 2 opens with a `kill -9`. Hold the distinction. Part 6 measures the level-3 window.

Here is the answer to the third question, and it is the bridge into the rest of the hour.

Two failures, and they are not the same failure.

Kill the process. Levels one and two are that process's memory, so they die with it. Level three is the kernel's memory and the kernel is still running. Those bytes get written back [V14]. Your process does not need to exist.

Pull the power. Now level three dies too, because it was RAM. Level four is the cache inside the drive. Whether that survives depends on power-loss protection, which is capacitors inside the drive. Most consumer drives do not have it, and that is my reading rather than a figure I can cite [speculative].

I am saying this now, before part two, for one reason. Part two opens with a kill demonstration. Students watch that demonstration, see the file survive, and conclude that flush was enough. It was enough against that test. My own note says the honest classroom demo is a power cut, not a kill [V14].

Nobody is pulling the plug in this room today. Instead, in part six, I measure how long level three holds your bytes. That is the window a power cut would have caught.

6 6. Predict now, and keep it for an hour

On screen:

Part 1 · Recap

The prediction for Part 6. You write it now, before Part 2, so you carry the question through the carried hour.

[code]

The question

The process has exited. How long until the kernel actually writes your bytes to the device?

Your options

Under a second · about five seconds · about thirty seconds · not until shutdown.

One of four. Write it in the corner of your page. Do not change it later [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

One more prediction, and then we go back to Lecture 2.

I am asking for it now rather than in forty minutes, and the reason is how memory works in a long lecture. You remember the start and the end of an hour, and the middle blurs. Part six is the middle. If you carry an open question through parts two to five, part six lands on a question you already own instead of arriving cold.

The program is on the screen, and it is the program every one of you has written. It opens a file, writes a hundred records with the default buffering, prints done, and exits. No flush call. No fsync call. That idiom "contains no durability step anywhere" [V4]. The full file is in the demo folder; the four lines here are all that matters for the question.

The process is gone. The kernel still has your bytes. How long does it keep them?

Four options. Under a second. About five seconds. About thirty seconds. Not until shut-down.

Write one. Corner of the page. I will ask you to read them out before I run anything, and I will hold you to what you wrote.

Do not go looking for the answer in the demo folder before then.

7 7. What dies with the process

On screen:

Part 2 · What dies with the process

Parts 2 to 5 are Lecture 2, finished. Same slides, same order, renumbered into today.

Part two. This is Lecture 2, finished, and it runs to part five.

The slides are last week's, carried across without a word changed. Same order, same sources. Only the part labels are today's, so that you track one set of numbers rather than two. Seven slides in this stretch are mine rather than last week's. Two are predictions I should have collected last week and did not. Three finish the close story, which last week asserted and never measured. Two are the bridges in and out.

One slide from Lecture 2 is not here. It was called what to do on Monday. That is a conference-talk ending and it makes no sense in a course. Its content was the assignment hand-off, and part ten does that properly.

Where we stopped last week was slide thirty-two, the buffer stack you have just drawn. The next slide says exactly that, and then we pick up at slide thirty-three with a program that prints nothing.

This is the longest part of today by a wide margin. I would rather finish it than summarise it.

8 8. Where Lecture 2 stopped

On screen:

Part 2

Last Tuesday ended after slide 32: the device's acknowledgement comes early [V20], and no tool in the room can see whether it lied [V5].

From here to the end of Part 5 the slides are Lecture 2's, as built. The clock ran out after slide 32; slide 33, the `os._exit` demonstration, is where we pick up [inference].

Your two predictions from Part 1 stay on your page: the level the bytes are in after `fflush()`, and what a power cut takes. The first demonstration answers a different question — what a process dying takes — so keep both apart [inference].

Last week we finished slide 32 and stopped. That slide said the device acknowledges a write early [V20]. No tool we can run in this room tells an honest flush from a fast lie [V5]. That is where we pick up.

Everything from here to the end of Part 5 is Lecture 2 as I built it. I have not rewritten it, and I have not renumbered the slides inside it. Only the part labels at the top changed, so that this lecture reads as one sequence.

You wrote two things down a few minutes ago. Keep them. The first demonstration is about a process being killed, which is a different failure from the machine losing power. Hold that distinction. Part 6 comes back to it.

9 9. Three lines that print nothing

On screen:

Part 2

A demonstration. You have the stack now, so predict this one.

[code]

To a terminal

Does hello appear?

Redirected to a file

What does the file contain?

Write down both answers before I run either one [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

Three lines. Import, print, exit [inference]. Nothing unusual anywhere.

Same method as part one: predict first, out loud. But this time I expect most of you to be right, and that is why this sits here rather than at the beginning.

Two runs. First in a terminal. Then redirected into a file, and I show you the file.

Take thirty seconds. Write two answers. Does hello appear on the terminal, and what is in the file?

Hands up if you think the terminal shows hello. Hands up if you think the file contains hello.

Now watch. Terminal first. And now redirected.

The file is empty. Zero bytes. If you predicted that, you did not guess. You used the diagram from the start of this part, and that is exactly what this demonstration tests.

10 10. Why the file is empty

On screen:

Part 2

The rule sits in the C standard, not in the kernel.

1

Mode follows destination

Output is "fully buffered if and only if the stream can be determined not to refer to an interactive device" [S5].

2

A terminal is interactive

Bytes go out per line [inference].

3

A file is not

Bytes wait [inference].

4

The exit skips cleanup

No write reaches the kernel [inference].

[S5] C99 draft §7.19.3. <https://busybox.net/~landley/c99-draft.html>

Here is the explanation, and notice where it comes from [S5].

The rule is in the C language standard. It is not an operating system behaviour and it is not a Python quirk. The standard says that standard output is fully buffered if and only if the stream can be determined not to refer to an interactive device.

Then the third line ends the process without running any cleanup. The standard explicitly permits termination paths that do not close files properly. The buffer is process memory. The process is gone. The bytes went with it.

Now the sentence that connects this back to section two. A trace of this program holds no write call at all. Not a failed one. Not a short one. None. The bytes never got as far as the operating system.

That is the top of the stack, proved to exist by removing it.

11 11. What this demonstration does not prove

On screen:

Part 2

It proves level one is real. It proves nothing about levels three and four.

"the honest classroom demo is a power-cut, not a `kill -9`. Killing the process loses only the runtime's buffer; the kernel still writes back." [V14]

1

What died

The process, and the level-one buffer with it [inference].

2

What survived

Anything already in the page cache [V14].

3

Wrong conclusion

"So `flush()` was enough." It was enough against this test [inference].

I have to bound that demonstration immediately, because it is easy to over-read.

Killing a process destroys that process's memory, and that is level one. Bytes the program gave to the operating system earlier sit in the page cache, and the operating system writes them back in its own time [V14]. Your process does not need to exist.

So this proves that level one exists and can lose your data. It proves nothing about levels three and four.

The honest test for the lower levels is a power cut, not a kill [V14]. Pull the plug and you find out what the page cache was holding. Nobody is doing that in this room today.

Why so careful? Because students test with a kill, watch the file survive, and conclude that the program flushed everything it needed to. That conclusion holds against a kill and not against a power failure. Same demonstration, correct conclusion — the difference is which level you claim it tested.

12 12. Predict: what will `ltrace` show that `strace` did not?

On screen:

Part 2

Before I run the second tracer. Two lines on your page [S14].

*It records the **library calls** a program makes [S31] — here, `fwrite` in the C library [inference]. `strace` sees `write` in the kernel instead [inference].*

1

The C program calls `fwrite` three times

How many `fwrite` lines does `ltrace` print?

2

The kernel saw one write

Does `ltrace -S` print that line too, or only the library calls?

The answer follows from the stack you drew twenty minutes ago [inference].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/> [S31] `ltrace(1)`. <https://man7.org/linux/man-pages/man1/ltrace.1.html>

Stop. Before I introduce the second tracer, two numbers.

One word first, so the question means something. The second tracer is called ltrace. It records the calls your program makes into shared libraries [S31]. That means fwrite in the C library, not write in the kernel [inference].

The program calls fwrite three times. How many fwrite lines will the library tracer print? Write a number.

Second. We already know the kernel saw one write. With the dash capital S option the tracer shows system calls as well. Will that write line appear?

This is not a trick. Both answers follow from the diagram you drew at the start of the hour [inference]. The library tracer watches the boundary above the library. The system-call tracer watches the boundary below it. Three calls go in at the top. One call comes out at the bottom.

If you can answer this, you have the takeaway from the whole of part two. You have it before I have shown you the tool.

Thirty seconds. Write both numbers.

13 13. `ltrace`: the same idea, one layer up

On screen:

Part 2

strace stops at system calls. ltrace stops at calls into shared libraries [inference].

What it does

It "intercepts and records the dynamic library calls" a process makes [S31]. It can also print "the system calls executed by the program" [S31].

Where it comes from

Juan Céspedes wrote it originally [S32]. The manual page names Petr Machata too [S31]. On Debian and Ubuntu you run apt-get install ltrace [S32].

How to use it

[code]

It runs on Linux only [inference].

[S31] ltrace(1). <https://man7.org/linux/man-pages/man1/ltrace.1.html> [S32] ltrace. <https://ltrace.org/>

Here is the second tracer. It makes the next slide checkable.

What it does. Its manual page says it intercepts and records the dynamic library calls the process makes, and that it can also print the system calls underneath [S31]. The page says its use is very similar to `strace` [S31]. So this is the idea you already have, one layer up [inference].

Where it comes from. The project page says Juan Céspedes wrote it originally [S32], and the manual page names Petr Machata as well [S31]. The same page gives the Debian and Ubuntu command [S32].

How you use it. With `-s` you get the system calls as well as the library calls, which is the option that matters here [S31]. Try it yourself: run both tracers on one program and watch the outputs disagree.

14 14. Which levels can you actually watch?

On screen:

Part 2

I promised three of four boundaries. Here is the tool each needs.

BOUNDARY	WHAT WATCHES IT	IN THIS COURSE
<i>App → library</i>	<i>ltrace sees it, strace does not [V10]</i>	<i>the tool above</i>
<i>Library → kernel</i>	<i>strace — where our tracer attaches [S1]</i>	<i>today</i>
<i>Kernel → device</i>	<i>a tracer for the kernel's block layer [V30]</i>	<i>elsewhere</i>
<i>Device → medium</i>	<i>no system-call trace shows whether the device flushed [V5]</i>	<i>no tracer</i>

On the first slide I said three of the four boundaries are observable. This is the slide where I show you which three, and why the fourth is not one of them [V5].

There are four boundaries in the diagram. Here is what each costs to see.

The top one needs the tool from the slide before this. My source states the pair directly: `printf` is visible to `ltrace` and `write` to `strace` [V10]. In our demo program the library call is `fwrite` instead. That is how you test takeaway two rather than believe it.

The second boundary is the one we have watched all hour.

The fourth is not observable from software at all.

So: three of these four you can watch, one of them with a tool you now know. The one you cannot watch is where data actually gets lost. I come back to that asymmetry in section six.

15 15. The same write, at three levels

On screen:

Part 3 · One write, three levels

Now that the levels have a reason to exist, walk down them with one small program.

Eight minutes. In section three I said the levels exist, and why. This section walks down them with a program you can compile yourself.

You write C in this programme, so I am not going to apologise for the C or detour through pointers. We read the core of it. I compile it in front of you. I run it under the tracer. Then we read its trace.

16 16. The same write, at three levels

On screen:

Part 3

LEVEL	PYTHON	C	BUFFERED?
Application	<code>f.write(b"x")</code> [S6]	—	yes [S6]
C library	—	<code>fwrite(...)</code> [S3]	yes [S3]
System call	<code>os.write(...)</code> [S20]	<code>write(...)</code> [S2]	no [inference]

The bottom row takes a **file descriptor** — the small integer naming an open file. You saw one already: the `= 3` in the trace line was a file descriptor [S1].

[S1] *strace(1)* — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html> [S2] *write(2)* — Linux man-pages. <https://man7.org/linux/man-pages/man2/write.2.html> [S3] *fflush(3)* — Linux man-pages. [S6] Python documentation, *open()*. [S20] *os* — Miscellaneous operating system interfaces, *os.write()*. <https://docs.python.org/3/library/os.html#os.write>

Read the table by column first, then by row.

The Python column and the C column are two routes to the same bottom row. Python's file objects hold a buffer at level one. C's *fwrite* fills a buffer at level two. Both call *write* in the end, and *write* is the system call.

Now read the last column, because it carries the point. Each level holds a buffer, or it passes the bytes straight down. A Python file object holds one. The *os.write* function does not, because it wraps the system call directly [S20]. The levels are not decoration. They are the mechanism you measured in part one.

The bottom row introduces a term I deferred until now. A file descriptor is the small integer a Linux program uses to name an open file in system calls. You have seen one: the *open* call returned three [S1]. It appears as the first argument of every *write* to that file.

Remember the word. In section five it is the first thing that differs.

17 17. Meet the program: `iodemo.c`

On screen:

Part 3

Nine lines of C. The first open fails on purpose, so the trace shows an error path.

[code]

It prints nothing, on purpose. A program that prints nothing adds no `write` lines of its own to the trace [inference].

Nine lines on the screen. The full file adds comments and error returns.

Step one opens a file that does not exist, and that failure is deliberate. If every call succeeds, the trace is success all the way down, and the most interesting difference between the two platforms never appears. I show you that difference in section five.

Step two opens the file we mean to write.

Step three writes fourteen bytes in three calls. The C library collects them in its own user-space buffer before any of them crosses into the kernel [S3] [S5]. Watch that number.

Step four closes the file and checks the return value of the close. Almost none of the C I have read does that [speculative]. I come back to why in section six.

The program prints nothing at all. My reason is the plain one. Every `write` line in the trace then comes from the three `fwrite` calls, and not from a print statement [inference]. A silent program also makes a trace that is easier to read [speculative].

I am compiling it now, with warnings on, and running it.

18 18. Three calls in the source. One in the trace.

On screen:

Part 3

3

fwrite calls in the source

14

bytes written in total

1

write call in the trace

The three `fwrite` calls go into one C library buffer. The manual calls that level "the user-space buffers provided by the C library" [S3]. The line to look for in the live trace, predicted from the source [inference]:

[code]

The `3` is the file descriptor. The `14` is all three `fwrite` calls, in one crossing [inference].

[S3] `fflush(3)` — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>

This is where takeaway two stops being a claim and becomes something on a screen.

Three `fwrite` calls in the source. Fourteen bytes. One `write` in the trace.

Point at the source. Point at the trace. There is no line-by-line correspondence between them, and there never was. What you write is a library call. What the operating system sees is a system call. Your source did not say how many system calls to make — the buffering policy in the library fixed that number [inference].

The line on the slide is a prediction. I wrote it from the source before I ran anything. Now watch the real trace. Check it against the prediction. If they differ, the prediction was wrong and the machine was not.

And notice when the write happens. Not at the third `fwrite`. It happens when `fclose` runs, because `fclose` hands the buffer down to the kernel. The call in your source and the call in the trace are not even at the same point in time.

19 19. `fwrite` takes a size and a count

On screen:

Part 3

Four arguments, and two of them multiply.

[code]

1

It returns a count

Items written, not bytes. The count equals bytes only when the item size is 1 [S21].

2

Fewer is legal

Below it, "a successful write() may transfer fewer than count bytes" [S2].

[S2] write(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/write.2.html> [S21] fread(3), which documents fwrite(3) — Linux man-pages. <https://man7.org/linux/man-pages/man3/fwrite.3.html>

A small thing that trips people. It costs thirty seconds now or twenty minutes when you try it yourself [speculative].

The call takes four arguments: a pointer to the data, the size of one item in bytes, the number of items, and the stream. In our program I passed one and six, meaning six items of one byte each. Six and one would have written the same bytes.

Why does it matter? Two reasons.

First, the return value counts items, not bytes. The manual says the count equals the number of bytes only when the item size is one [S21]. Write six items of one byte. You get four back, and you wrote four bytes. Write one item of six bytes. You get zero back, and you wrote nothing.

Second, fewer is legal. The system call underneath may transfer fewer bytes than you asked for [S2], and the library call may report fewer items than you asked for [S21]. Two different contracts, one habit. Read the return value.

20 20. Two operating systems, side by side

On screen:

Part 4 · Two kernels

Read as a comparison. Try the tools yourself afterwards.

Twelve minutes, and some of you waited for this section.

A warning about what this is not. It is not a demonstration that one platform is better. It is not a tutorial on installing a Windows tracer. It is a reading exercise on two pieces of output. The lesson is a structural difference between the systems, not a difference in formatting.

I also need to be careful about what evidence I have. I will tell you exactly what I am showing you, where it came from, and what it cannot do.

21 21. Process Monitor: what the kernel did

On screen:

Part 4

It reports filesystem and registry activity [S18], which is a different cut than the system-call boundary [inference].

What it does

It shows "real-time file system, Registry and process/thread activity" [S18]. That is a filesystem-and-registry view, not a list of system calls [inference].

Where it comes from

Mark Russinovich wrote the page, in Microsoft's Sysinternals documentation [S18]. The tool "combines the features of two legacy Sysinternals utilities, Filemon and Regmon" [S18]. You download it. You can also run it from Sysinternals Live [S18].

How to use it

[code]

[S18] Process Monitor. <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>

Two Windows tools, one after the other.

This is the first. Microsoft describes it as showing real-time file system, Registry and process activity [S18]. That is a filesystem-and-registry view of the machine, and it is not a list of the system calls a program made [inference].

Where it comes from. Mark Russinovich wrote the page, in Microsoft's Sysinternals documentation [S18]. Microsoft says the tool combines two older utilities, Filemon and Regmon [S18]. You download it, or run it from Sysinternals Live without installing anything [S18]. The same page also links a Procmon for Linux [S18].

How you use it. It captures everything at once, so you filter by process name first. Then you read the Operation column. I expect `CreateFile` and `WriteFile` rows for our program [inference]. Check me on your own machine.

22 22. NtTrace: a tracer for the Windows Native API

On screen:

Part 4

It watches the native API, "the interface between the application space and the OS kernel" [S10].

What it does

A "simple trace facility for the Windows Native API", "roughly equivalent to strace on Linux" [S10]. It attaches as a debugger, so it "needs no driver and no administrator rights" [V21].

How to use it

[code]

Try both yourself on `iodemo.exe` : Process Monitor shows what the filesystem did, NtTrace shows what the program asked for [inference].

[S10] NtTrace — Roger Orr. <https://rogerorr.github.io/NtTrace/> [S33] NtTrace source. <https://github.com/rogerorr/NtTrace>

And here is the second tool, the one we read today.

What it does. Its project page says it provides a simple trace facility for the Windows Native API, and that it is roughly equivalent to `strace` on Linux [S10]. That comparison is the tool author's own, so you should know whose sentence it is. The same page says the tool uses the Windows debug interface to intercept the returns from the native API [S10]. The consequence, from my reading note: it needs no driver and no administrator rights [V21]. That one choice is why it runs on your own laptop at all.

Where it comes from. Roger Orr wrote it and still maintains it: the project page lists releases from 2011 to February 2026 [S10]. The source is on GitHub under the MIT licence [S33].

How you use it. The page's own example filters on the substring `File`, which is the form we want [S10]. One honest word: this tool is one person's work, and Microsoft calls those entry points "subject to change from one release of Windows to another" [S9].

Run both tools on the same program. That is the fastest way to see that they watch different layers. Process Monitor reports filesystem events. NtTrace reports the native calls the program made [inference].

Be precise, because I could overstate it. NtTrace is not at the same attachment point as the Linux tracer. The Linux tracer sits at a documented kernel boundary. NtTrace sits one level above the kernel, in a user-mode library. That gap is the lesson I want you to take from this section [S10].

Neither of today's tools is what people run in production, and I come back to that at the end.

23 23. Three Windows words you will need

On screen:

Part 4

[code]

1

Win32

The documented interface a Windows program calls. The 32 is history, not a width [inference] [S22].

2

The native API

The layer below Win32, inside ntdll.dll. Microsoft calls it "internal to the operating system" [S9].

3

The object manager

The kernel component that names open objects. Its paths begin ??\ [V22].

[S9] winternl.h — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation> [S22] Windows API index — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list>

Three words, thirty seconds, and then the Windows trace makes sense [speculative]. The picture is the same stack you drew in Part 1, seen from the side: your program, two libraries, then the boundary into the kernel.

Win32 is the documented interface a Windows program calls. It is the Windows equivalent of the C library plus the POSIX calls. It is the interface the documentation tells you to use. The name is historical. Microsoft renamed the same thing the Windows API. The new name "more accurately reflects its roots in 16-bit Windows and its support on 64-bit Windows" [S22]. So the 32 in the older name tells you nothing about the width of a program or of a trace [inference]. The trace you see next came from a sixty-four bit process [S36], and the name Win32 still applies to it [inference].

The native API is the layer below Win32. It lives in a library called ntdll, and it is the last user-mode code before the kernel. Microsoft's own documentation calls it internal to the operating system [S9]. NtTrace attaches at this layer, which is why its lines start with Nt [S10].

The object manager is the kernel component that names open things — files, processes, events. Its paths look different from the paths you type. You see one later in this section, in a trace of the Windows command shell [S34].

24 24. Predict: the same program, on Windows

On screen:

Part 4

Before the Windows capture goes on the screen. Two answers, on paper [S14].

1

The three fwrite calls

How many write calls does the Windows trace show — three, or one?

2

The deliberate failure

Linux reported -1 ENOENT. How many error codes does Windows report for the same failed open?

The first answer tests whether buffering is a C library property or a platform property. The second one has a surprise in it.

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

One more prediction, and this is the one I most want you to commit to.

Same source file. Same three fwrite calls. Compiled with the Microsoft compiler and traced on Windows.

First question. How many write calls does the Windows trace show? Three, or one?

Think about where the buffering happened on Linux. It happened in the C library, and the C library is not the kernel. So the answer tells you whether you learned a Linux fact or a C fact last week.

Second question, and this one is harder. The first fopen fails on purpose. Linux answered minus one and the name ENOENT. One error, one namespace. How many error codes does Windows report for the same failure?

I expect most of you to say one [speculative]. Write down what you think and I will show you the capture [speculative].

Thirty seconds.

25 25. A real Windows trace

On screen:

Part 4

You can already read this. The anatomy matches the Linux line, and the vocabulary differs [inference].

[code]

iodemo.exe under NtTrace, 2026-09-15 — the same four steps as the Linux trace [S36] [S37]. Ignore every argument except the handle, the file name and the return value [inference].

[S36] Our NtTrace64 capture of iodemo.exe, 2026-09-15 — demo/iodemo-c/NtTrace64-IODEMO-Output.txt [S37] The container capture of iodemo, 2026-09-15 — demo/iodemo-c/README.md

This is our own program on Windows. I traced `iodemo.exe` with `NtTrace64.exe` on 15 September 2026, and I traced the same program on Linux the same day [S36] [S37].

Read them as the four steps from part four [S36].

Name, arguments in parentheses, return value after the arrow. Same anatomy as the one Linux line I made you read.

One thing does differ, so I say it now. Here `=> 0` means success, and a return value starting `0xc` is a failure code [S36] [inference].

The three dots mark arguments I cut [S36].

The capture runs to 45 lines [S36]. These are the last four traced calls. Every call above them is the loader's, opening its own files before `main` ran [S36] [inference].

The first call wraps over two printed lines, so count calls and not lines. Call one is the deliberate failure, and its return value carries two error codes [S36]. Call two opens `iodemo.out` and returns handle `0x124` [S36]. Call three writes `Length=0xe` bytes, which is fourteen — three `fwrite` calls in one crossing [S36] [inference]. Call four closes the handle [S36].

One argument looks unfamiliar. `ObjectAttributes=0x58:"iodemo.out"` means the program named the file relative to an open directory [S36] [inference]. So neither of our two opens carries the object-manager prefix [S36]. The loader's calls above do carry it [S36]. The one I show you comes from the shell [S34].

26 26. A handle is not a file descriptor

On screen:

Part 4

First difference, visible in the first argument of almost every line.

Linux

A file descriptor. A small integer indexing a per-process table. Our trace shows = 3 [S37].

Windows

A handle. An opaque value naming an open object. Our trace shows FileHandle=0x124 [S36].

Same job in a trace, different mechanism in the system. Do not use one word for the other [inference].

*[S36] Our NtTrace64 capture of iodemo.exe, 2026-09-15 — demo/iodemo-c/NtTrace64-
IODEMO-Output.txt [S37] The container capture of iodemo, 2026-09-15 — demo/
iodemo-c/README.md*

First difference, and it sits in the first argument of nearly every line on both systems [S36] [S37].

One program, one file, two systems. Linux answered `3`, and Windows answered `0x124` [S36] [S37].

On Linux you get a file descriptor. A small integer indexing a table that belongs to your process. Three, four, five. You can reason about the numbers.

On Windows you get a handle. An opaque value that names an open object, and the object need not be a file. It might be a process, an event or a registry key.

In a trace they occupy the same position and do the same job, which is why it is tempting to use one word for both. Do not. In this course I hold the two words apart. If you write handle in a Linux assignment, I will ask which system you meant.

This is a small thing. I spend a slide on it because it is the cheapest instance of today's platform lesson: the same job, two mechanisms, visible in one argument.

27 27. Two error namespaces in one return value

On screen:

*Part 4**Second difference, and it is the layer boundary showing up where nobody expects it.**[code]**1**First number**An NTSTATUS, the native layer's code [S36] [inference].**2**Second number**The Win32 error it translates into [S36] [inference].**3**Linux**One errno: -1 ENOENT [S37].**[S36] Our NtTrace64 capture of iodemo.exe, 2026-09-15 — demo/iodemo-c/NtTrace64-
IODEMO-Output.txt [S37] The container capture of iodemo, 2026-09-15 — demo/
iodemo-c/README.md*

Second difference, and this is the one I find genuinely interesting [speculative].

That return value is our own program's first line, the `fopen` I made fail on purpose [S36].

Look at it. It carries two error codes at once. The first is the native layer's own code. The second is the Win32 error the first translates into, with the message text you would actually see.

Two namespaces, in one return value, from one call.

Linux has one. The same failed open returns `-1` and the name `ENOENT` [S37]. That number is the error, and there is nothing to translate, because only one layer names errors.

Why does Windows have two? Because there genuinely are two layers there, and both are real interfaces with their own error vocabularies. You are looking at the seam between them, visible because the tracer sits below the translation and reports both sides.

This is why the demo program opens a file that does not exist. Without a failure this line never appears, and a clean run hides the seam between the two layers.

28 28. The stable boundary sits at a different level

On screen:

Part 4

Third difference, and the other two are symptoms of it.

In our trace of the Windows command shell a path reads "\??\C:\WINDOWS\system32\c_852.nls" [S34]. That prefix "is the object-manager namespace, which a Win32 program never types" [V22].

Linux

[V8]

The system call interface is the public contract.

Syscall

Windows

[S9]

Native entry points are "subject to change from one release of Windows to another".

Win32

[S9] winternl.h — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation>

Third difference, and the other two are symptoms of this one [inference].

Look at the path on this slide. It comes from the shell's trace [S34]. Our own program's two opens name their file relative to an open directory, so neither of them carries the prefix [S36].

The path starts with a backslash, two question marks, a backslash. No Win32 program ever types that string. It is the name of the file in the object manager's namespace, which sits below the interface a Windows programmer uses. The tracer is showing you a layer programs do not normally see.

Microsoft says so in its own documentation. The native entry points are internal to the operating system and subject to change between releases [S9]. That is a first-party statement.

On Linux the opposite holds. The system call interface is the public contract. You may write a system call number down and rely on it.

Now the careful sentence, and it is the takeaway. By nominate I mean one thing: the vendor documents that layer as the one it will not break. Both systems are stable at the layer they nominate, and the disagreement is about which layer that is. Everything else follows from that single placement, including where a tracer can attach.

29 29. Where the boundary sits is contested history

On screen:

Part 4

Two readings survive the evidence. I will not pick one for you.

1

Reading A: deliberate

POSIX, OS/2 and Win32 each reached NT through the native API [S19].

2

Reading B: vestige

A reported POSIX subsystem that was "never really completed" [S15].

3

Linux had no choice

It "had to make the kernel boundary itself the stable interface" [V38].

[S15] The sad history of the Microsoft POSIX subsystem — Reiter (2010). <https://brian-reiter.org/2010/08/24/the-sad-history-of-the-microsoft-posix-subsystem/> [S19] Inside Native Applications — Russinovich (2006). <https://learn.microsoft.com/en-us/sysinternals/resources/inside-native-applications>

I said the placement was a decision. Let me be more careful, because the evidence supports two readings and I would be overstating if I gave you only one [S15] [S19].

Reading A is design. NT had three operating environments: POSIX, OS/2 and Win32 [S19]. Each one served its own clients through its own interface. Each one reached NT through the native API [S19]. The source does not say where those environments ran, so neither do I. On that reading the split is deliberate. An unstable lower layer then follows from it [inference].

Reading B is vestige. A POSIX subsystem did exist, and one practitioner account says it was built to satisfy United States government procurement rules and never finished [S15]. On that reading the split is what remained after the plan was abandoned.

Both accounts are in the literature, and I am not going to settle it in a practicum. The mechanism is settled. The intent is not.

Linux is the clearer case. It was a kernel built against a separately developed userland it did not control [V38]. It could not update the C library in lockstep, so it had to make the kernel boundary itself the stable one [V38].

30 30. Linux is the outlier, not Windows

On screen:

Part 4

Only Linux promises that the raw system call interface stays the same [V9]. Say different, never worse [inference].

1

Who makes no such promise

FreeBSD, OpenBSD, NetBSD, Solaris/illumos, macOS [V43].

2

Not promised is not broken

FreeBSD keeps old programs running through compatibility layers; it never writes the promise down [V36].

3

Why Windows and macOS can

They ship their own C library, so the library is the stable surface and the kernel boundary can move [V38].

Now the correction to the conclusion you were probably forming [speculative].

After the last two slides it is tempting to think Windows is the odd one out. It is not. My source names the list: FreeBSD, OpenBSD, NetBSD, Solaris/illumos and macOS all behave like Windows here [V43]. None of them promises that the raw system call interface stays the same. Linux does. Linux is the exception [V9].

Do not read that list as five broken systems [V36]. FreeBSD keeps old programs running through compatibility layers. It holds the interface in practice. It just does not write the promise down [V36].

Why can Windows and macOS skip the promise? Because each ships its own C library. Programs call the library, the library is the stable surface, and the kernel boundary below it is free to move [V38]. Linux could not do that, because its C library was built by somebody else.

In this course you say the platforms are different. You do not say one is worse. If you write that, I will ask which property you are ranking them on.

31 31. One program, two kernels

On screen:

Part 4

One program, two systems, traced on one day.

STEP	LINUX [S37]	WINDOWS [S36]
<i>failed open</i>	<code>openat(... "no-such-file.txt" ...) = -1 EN-OENT</code>	<code>NtCreateFile(... "no-such-file.txt" ...) => 0xc0000034 [2 '...']</code>
<i>open</i>	<code>openat(... "iodemo.out" ...) = 3</code>	<code>NtCreateFile(... [0x124] ... "iodemo.out" ...) => 0</code>
<i>write</i>	<code>write(3, "one\ntwo\nthree\n", 14) = 14</code>	<code>NtWrite-File(FileHandle=0x124, ... Length=0xe ...) => 0</code>
<i>close</i>	<code>close(3) = 0</code>	<code>NtClose(Handle=0x124) => 0</code>

Same program, same fourteen bytes, one write on each side. Hold the program fixed, and what is left is the two systems [inference].

[S36] Our NtTrace64 capture of *iodemo.exe*, 2026-09-15 — *demo/iodemo-c/NtTrace64-IODEMO-Output.txt* [S37] The container capture of *iodemo*, 2026-09-15 — *demo/iodemo-c/README.md*

This is the slide the section was built for [speculative].

Read across the rows, not down. Each row is the same step of the same source file [S36] [S37].

Row one is the deliberate failure. The Linux call is `openat`, which is `open` with a starting directory [inference]. It returns `-1` and one error name [S37]. Windows returns a native code and the Win32 error it translates into [S36] [inference].

Row two is the open, and the two columns are not symmetric. Linux returns the file descriptor `3` as the return value [S37]. Windows returns the status `0` and hands the handle `0x124` back through an argument [S36] [inference].

Row three is the write. Fourteen bytes on Linux, `Length=0xe` on Windows [S36] [S37]. Three `fwrite` calls in the source, one call in each trace [inference].

One number to reconcile, from part two. Seven write operations there, one here. The other six were the Linux loader's [S37].

Now the caveat, and it is about the Windows tool rather than about this pair. NtTrace is one author's work [S10]. Microsoft documents those native entry points as subject to change between releases [S9]. So read the Windows column as the shape of the boundary, not as an interface you may depend on [inference].

My own note on the sample I taught from says "It is not a matched pair." [V23] This one is. Both captures ship in this deck, and you can rerun them yourself.

32 32. What we have not looked at

On screen:

Part 5 · Buffering moves your errors

One question I will not answer, and the work that follows from it.

Six minutes left, and I am not going to spend them listing topics.

I will put one question back on the screen. I will not answer it. Then I show you two consequences of the stack. You already have everything you need to see them.

33 33. `write()` returned success. The bytes are not on a disk.

On screen:

Part 5

I built the whole hour towards this sentence. I leave it unresolved.

The manual says it plainly: "A successful return from `write()` does not make any guarantee that data has been committed to disk" [S2].

Saltzer, Reed and Clark gave the general form in 1984. A lower level may offer "an incomplete version of the function... useful as a performance enhancement" [S11].

So when is it actually safe? That is the durability lecture's question. I leave it open today.

[S2] `write(2)` — Linux man-pages. [S11] End-to-End Arguments in System Design — Saltzer, Reed and Clark (1984). <https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf>

Read the first quote again. It comes from the manual page for the most basic write call there is. It says a successful return guarantees nothing about a disk [S2].

That should now be obvious rather than alarming, because you have the stack. A successful write means the bytes reached the next level down. Which level depends on which call you made.

The second quote is there so you do not think this is a bug someone will fix. Saltzer, Reed and Clark wrote it in 1984. A lower level may offer an incomplete version of a function as a performance improvement. The complete version needs the application at the end points. Every level in our diagram is an instance of that.

So: when is it actually safe? I am not answering today. I want you to leave with the question open, because a programmer who believes it is already answered writes no durability step at all.

34 34. Buffering moves your errors

On screen:

*Part 5**The error arrives somewhere other than where you caused it.**1**Why it moves**The bytes only reached a buffer [inference].**2**Where it arrives**Where the library flushes, so at fclose [inference].**3**The rule**Check the return value of the call that flushes [inference].**4**The idiom hides it**"the most common durability idiom in every mainstream language... contains no durability step anywhere" [V4].*

Here is the practical consequence, and it will cost you a debugging afternoon at some point [speculative].

Your fwrite returned success because the bytes reached a buffer. A full disk, an unplugged device, an exceeded quota — no error from any of those has reached the library yet, and none can [inference].

That is why the demo program checks the return value of fclose. Almost none of the C I have read does [speculative]. That single check is the difference between having written fourteen bytes and believing you did.

The last row is worth saying slowly. The most common way to write a file in every mainstream language contains no durability step anywhere [V4].

Python differs here. A Python with block does raise if the close fails. The two languages differ in what they force you to notice, which is the property I am ranking them on [inference].

This is error reporting, not error recovery. Recovery belongs to the durability lecture.

35 35. Predict: all your writes succeeded — can the close still fail?

On screen:

Part 5

Yes or no, on paper. Then a second question: if it fails, is the descriptor still yours?

1

Every `write()` returned the byte count

The program did nothing wrong that it could see.

2

Now `close()`

Can it return -1? On what kind of filesystem?

3

If it did

Is the file descriptor number still open, closed, or unknowable?

Two answers, written down [S14].

Two predictions, both yes-or-no, both on paper.

First. Every write in your program returned the number of bytes you asked for. Can the close still fail? Commit to yes or no.

Second. Suppose it did fail. Is the descriptor still open, already closed, or can you not tell? Write one of the three.

These are not trick questions. Most of this room will get the first one wrong, and for a good reason. Nothing in the buffer stack we drew says that a close does any work [inference].

36 36. Why a close can fail when every write succeeded

On screen:

Part 5

The kernel deferred the write's error. The close surfaces it [S4].

1

Deferred

"errors on a previous write(2) operation are reported only on the final close()" [S4].

2

Where it bites

"silent loss of data ... with NFS and with disk quota" [S4].

3

The mirror

"A successful close does not guarantee that the data has been successfully saved to disk" [S4]. Part 6's question.

Against the stack: write() said done at level 3 [S2]. close() can still refuse [S4].

[S4] close(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/close.2.html>

So, the first answer: yes, it can fail, and the manual page tells you when [S4]. The write's error was deferred. The kernel took your bytes into the page cache and said done. Later it tried to push them on, and the network file system or the quota said no. Close is the last call in your program that can still report that.

The manual gives two examples, NFS and disk quota. In both the check happens after the write returned. Most of you will not hit them on a laptop. That is exactly why the habit of checking close matters. The code that works on your machine loses data on the server.

Read the mirror sentence too. A successful close does not mean the data is on disk. Close does not sync. So checking close is necessary and not sufficient, and Part 6 is about the sufficient part.

37 37. Observe: close it twice, and watch what the second close hits

On screen:

Part 5

demo/close/ — one descriptor, two closes, both platforms [S39].

[code]

The first close released the number, error or not: Linux "always releases the file descriptor early in the close operation, freeing it for reuse" [S4]. So a retry closes nothing — or, with threads, "a reused file descriptor from another thread" [S4].

Under a debugger `CloseHandle` "will throw an exception if it receives ... a handle value that is not valid ... This can happen if you close a handle twice" [S38]. NtTrace is a debugger, so the second close raises [S39].

[S4] `close(2)`. <https://man7.org/linux/man-pages/man2/close.2.html> · [S38] `CloseHandle`. <https://learn.microsoft.com/en-us/windows/win32/api/handleapi/nf-handleapi-closehandle> · [S39] `demo/close/README.md`. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/close/README.md>

Now the second answer, and this one we can watch. A ten-line program opens a file, writes, closes, and closes the same descriptor number again. That second close is exactly what a retry would do.

On Linux the trace shows it. Close of three returns zero. Close of three again returns minus one, bad file descriptor [S39]. The kernel released the number in the first call. The manual says it always does that early, and that the steps which can fail happen afterwards [S4]. So a failed close leaves you nothing to retry. Another thread may already hold that number.

On Windows the second `CloseHandle` fails with error six, invalid handle [S39]. Here is the twist. Microsoft documents that under a debugger the same call throws an exception instead [S38]. NtTrace is a debugger. So the traced Windows run raises rather than failing quietly. The operating system says, as loudly as it can, that a double close is a bug.

Put the two answers together and the rule from three slides back follows. Check the return value of close. Never retry it. Treat a failure as data you may have lost.

38 38. Never retry a failed close

On screen:

Part 5

The obvious repair is wrong, and the manual page says so [S4].

"Retrying the close() after a failure return is the wrong thing to do, since this may cause a reused file descriptor from another thread to be closed" [S4].

A retry closes whatever now sits at that number, which may be another thread's file [S4].

Report the error and stop. Part 6 shows the same rule one level down, at `fsync`, and the data loss that taught it [V40].

[S4] close(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/close.2.html>

Now you check the return value of `close`. You will want to do the obvious thing with it, and the obvious thing is wrong [S4].

Do not retry. The manual page says why, and the reason is specific. After a failed `close` the descriptor may already be gone. If another thread has opened a file meanwhile it may have been given that same number. Your retry closes somebody else's file.

The transferable rule: when a failure leaves you unable to tell what state you are in, stop the program and report it. Continuing hides the damage and does not repair it.

The databases learned this the hard way, one level down, at `fsync`. That story has its own slides in Part 6, after you have seen what `fsync` promises [V40].

39 39. Networking and IPC are this same mechanism

On screen:

Part 5

Not new subjects. The same objects and the same calls, wearing different names.

1

A socket on Linux

It is a file descriptor, and Linux makes that contract binding [V28].

2

A socket on Windows

An opaque SOCKET, and Windows makes Winsock the contract [V28].

3

A pipe

A file descriptor too, so the same calls reach it [V29].

Depth belongs to the next lecture. I want to stop you filing networking under "something else".

Thirty seconds, and then I stop naming things [speculative].

When you meet networking in this course it will not be a new mechanism. On Linux a socket is a file descriptor. The same read and write calls work on it. Sockets, pipes and terminals are all file descriptors, which is why one program can wait on all of them at once [V29].

On Windows a socket is an opaque type rather than a file descriptor. The binding contract is Winsock, the Windows sockets interface, rather than the file-descriptor contract [V28]. That is section five's pattern showing up again in a place you did not expect it [V8]. Same job, different mechanism.

40 40. Honest limits

On screen:

Part 5

What today's tools cannot see.

•

The device level

[V5]

Not observable from software: no trace can check it.

•

Asynchronous I/O

[V11]

The tracer sees the setup, then no I/O [inference].

Here is what I did not establish today.

The device level is not observable from software [V5]. I said it in section three and I repeat it here. It is the limit on today's thesis, and it is where data is actually lost [V5].

Asynchronous I/O breaks the model you have just learned, and it breaks it interestingly. A modern Linux interface moves submission into shared memory rings [V11]. One option starts a kernel thread. That thread polls the rings. Steady-state submission then makes no system calls at all. The tracer sees the setup and then nothing. The boundary you just learned to watch is already moving.

41 41. Honest limits, continued

On screen:

Part 5

And what today's tools cost.

•

strace in production

[V25]

Even the cheaper --seccomp-bpf mode [V39] "should therefore not be used on production systems!"

•

Mapping, crash dumps

—

Each has its own lecture.

And the tools. The tracer has a second and cheaper mode, selected with the option `--seccomp-bpf`. It hands the kernel a filter, so the tracer stops only for the calls you asked about. On the workload of rebuilding strace itself, plain strace costs 3.06 times the untraced run and that mode costs 1.44 times [V39]. A strace contributor disqualifies it anyway. The mode "still stops at each syscall of interest and should therefore not be used on production systems!" [V25]. Production uses a different mechanism, in-kernel programs rather than a stop at every call: eBPF on Linux, and event tracing on Windows [speculative]. I have no per-call figure for either, so I am not giving you one. Today's tools watch the layer I am teaching, which is why I use them.

Finally, memory mapping and crash dumps. Each has a lecture.

42 42. Lecture 2 ends here. The question does not.

On screen:

Part 5

write() returned success and the bytes are not on a disk [S2] — so when is it safe? Part 6 measures it.

Part 5 named the question and the honest limits of the tools. Your prediction for Part 6 has been on your page since Part 1: under a second, about five, about thirty, or not until shutdown [inference].

That was the end of Lecture 2. It left one question open, and you predicted an answer to it an hour ago. A successful write is no guarantee that the data is on disk [S2]. Look at your page. Part 6 runs the program and we watch the kernel finish the job.

43 43. So when is it safe?

On screen:

Part 6

The question I left you with last week. Your prediction is already on your page.

Part six. This is the part I care most about today.

Last week ended on a question and I refused to answer it. Write returned success and the bytes are not on a disk, so when is it actually safe?

You wrote a prediction an hour ago. Under a second, about five seconds, about thirty seconds, or not until shutdown. Do not change it now.

Before I run anything, I want to hear three of you. Read out what you wrote and say why.

Then we measure it, here, on this machine, and we find out together who was right.

44 44. The kernel's own count of what it has not written

On screen:

*Part 6 · Durability**No tracer here. The kernel publishes the number itself, and we watch it fall.**[code]**Dirty**File data the kernel holds and has not written to the device yet. A page in this state is a dirty page [V78].**Writeback**Data the kernel is writing to the device right now [V78].**It is a machine-wide counter, so we watch it rise when `save.py` runs and fall when the kernel is finished [S52].**[S52] [demo/durability/README.md](https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md) — captured 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md>*

This demonstration uses no tracer at all, and that is deliberate.

Last week you learned that a tracer changes the program it watches. So for this measurement I am not watching the program. I am watching the kernel, and the kernel publishes the number itself in a file called `proc meminfo`.

Two fields matter. Dirty is file data the kernel is holding and has not written to the device. Writeback is data on its way out right now [V78].

The word for a page in that state is a dirty page. It is the kernel's own word, and you will see it in the output.

One honest caution before I run it. This counter belongs to the whole machine, not to our file. Something else could write during our window. So watch the shape rather than the absolute number. It rises when `save.py` runs, and it falls when the kernel is done.

Running it now.

45 45. Observe: the program finished at zero seconds

On screen:

Part 6 · Durability

```
[code]
```

There it is. Read it from the top.

The first two lines are the kernel's own timers, printed before the run. I come back to them on the next slide but one, because they are the interesting part.

Then the samples. At time zero the machine is holding about one mebibyte of dirty data. That is the baseline.

Still at time zero, the figure jumps by four mebibytes. Our hundred records. And read the annotation on that line carefully. `Save.py` has already exited. The process is gone. It printed done and returned to the shell, and four megabytes of your data are sitting in the kernel's memory.

Now watch the next five lines. Half a second. Two and a half seconds. Four seconds. The number does not fall. It creeps up slightly, because the machine is doing other things.

Five seconds. It drops back to baseline. The kernel has written your bytes.

Five seconds after your program said done [S52]. Hands up who wrote five seconds. Hands up who wrote thirty.

One thing on that last line before somebody asks, because it does not do what the watcher says it will. The watcher samples until the count returns to where it started. It finished at six hundred and seventy-six kilobytes, against a baseline of one thousand one hundred and seventy-six [S52]. It went below. That is the counter being machine-wide rather than ours: something else on the machine finished writing back during our five seconds too. Read the fall, not the final figure.

46 46. Five runs, three sizes

On screen:

Part 6 · Durability

One run is an anecdote. Here are all five, including the one that disagrees.

WRITTEN	RUNS	THE KERNEL'S DONE, AFTER THE PROGRAM'S
$100 \times 40 \text{ KiB} = 4 \text{ MiB}$	3 (two on overlayfs, one on plain ext4)	5.0 s, 5.5 s, 5.5 s
$100 \times 64 \text{ B} = 6 \text{ KiB}$	1	5.5 s
$1000 \times 40 \text{ KiB} = 40 \text{ MiB}$	1	0.5 s

Four of the five runs land at five seconds, and file size barely matters between six kilobytes and four mebibytes [S52]. The forty-mebibyte run went out ten times sooner, which the next slide explains.

[S52] [demo/durability/README.md](https://on-drasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md) — five runs, captured 2026-09-20. <https://on-drasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md>

I ran it five times before this lecture, and I am showing you all five [S52].

Four of them land in the same place. Five seconds, five and a half, five and a half, five and a half. The file size made almost no difference between six kibibytes and four mebibytes. That is the result.

The fifth run is the one worth your attention. Forty mebibytes went out in half a second. Ten times sooner, for ten times the data.

If you have just thought "so more data is safer", stop. That is exactly the wrong conclusion and I explain it on the next slide.

Two limits on this table. Every figure is one machine, one kernel, one filesystem. And two of the three big runs use the container's overlay filesystem, not a plain ext4 partition [S52]. An overlay filesystem stacks on top of another filesystem. I repeated one on plain ext4 and got the same answer [S52]. That is why I am willing to put the number on a slide.

47 47. Five seconds in memory that belongs to nobody

On screen:

Part 6 · Durability

The program's done and the kernel's done are different events, five seconds apart.

1

Where the bytes were

In the page cache, level 3 of your drawing. That memory is the kernel's [V78].

2

What set the delay

A kernel timer, not your program and not Python [V78].

3

What a power cut takes

Everything in that window [V14].

The return value was correct. It reported that the kernel had accepted the bytes [V50].

Now the explanation, and it is short because you already have the model.

Where were the bytes for those five seconds? In the page cache. Level three of the diagram you drew at the start of the hour. That memory belongs to the kernel and to no process, which is exactly why killing the process did not lose it [V78].

What set the five seconds? Not you. Not Python. Not the C library. A kernel timer did, under a rule your program cannot see and did not set [V78].

And this is the answer to the third recap question. You asked what a power cut loses. Now you know the size of the window. On this machine, for an ordinary file, about five seconds [V14].

Say the last line with me. The return value was correct. Write returns when the kernel has accepted responsibility for the bytes, not when the work has happened [V50]. It reported a handoff. You read it as a report about the disk.

That is the whole lecture, and we are only in part six.

48 48. Which rule fired, and how we know

On screen:

Part 6 · Durability

The setting a student reads first is not the one that applied.

dirty_expire_centisecs

This machine: 3000, so 30 s [S52]. It marks when data is "eligible for writeout" [S68].

dirty_writeback_centisecs

This machine: 500, so 5 s [S52]. It sets how often the "kernel flusher threads ... wake up" [S68].

The 40 MiB run

Above dirty_background_ratio, "writeback starts sooner" [V48].

Writeback is the kernel copying dirty pages out. Which rule fired here is my reading [inference].

[S68] *admin-guide/sysctl/vm.rst*. <https://www.kernel.org/doc/html/latest/admin-guide/sysctl/vm.html> [S52] *demo/durability/README.md*. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md>

This is the slide where I tell you what the measurement does not prove.

A student who wants to know when writeback happens searches, and finds dirty expire centisecs first. The kernel page says it sets when dirty data is old enough for writeout [S68]. Our machine read three thousand, which is thirty seconds [S52]. Our pages went out in five. So the setting a student meets first is not the one that applied.

Our machine read the second setting as five hundred, which is five seconds [S52]. The page says this setting controls how often the flusher threads wake up [S68]. Five seconds is also our answer. That is a strong match.

And the forty megabyte run went before any timer, because it crossed a memory threshold. Above dirty background ratio, writeback starts sooner [V48]. That is why more data went out faster.

Here is the honest sentence. Writeback has several competing rules. I am inferring which one fired from the timing, and this demonstration cannot show me the rule directly [inference]. If you want certainty you need a kernel tracer, and that is a different course.

The lesson survives either way. The kernel's done is on the kernel's clock and you did not set it.

49 49. Visibility is not durability

On screen:

Part 6 · Durability

The objection you are about to make, and the answer to it.

Visibility

After a successful write(), a later read returns the bytes. Another process reading the file sees them too. This is a coherence property of the page cache [V45].

Durability

The bytes survive a power loss. This is a property of the medium [V45].

My own note calls confusing the two "the single most common error in this area" [V45].

So `cat results.csv` returning your data proves visibility. It proves nothing about the disk [inference].

Somebody in this room is thinking: but I can open the file and read it back, so it must be saved.

You are right that you can read it back. POSIX guarantees it. After a successful write, a later read returns the bytes, and a read from a different process returns them too [V45].

That property has a name. It is visibility, and it is a coherence property of the page cache. Of course you can read the bytes back. They are sitting in the kernel's memory and the kernel serves your read from there.

Durability is a different property. It means the bytes survive a power loss, and it is a property of the medium [V45].

My note on this says confusing the two is the single most common error in this whole area, and I have made it myself [V45].

So here is the test that does not work. Running `cat` on the file proves visibility [inference]. It tells you nothing at all about whether the disk has your data. During those five seconds, `cat` would have shown you every byte.

Two words, held apart. Visibility. Durability.

50 50. Predict: what does one `fsync` per record cost?

On screen:

Part 6 · Durability

Before the counts. Same program, same hundred records, four ways [S14].

Measured

One `f.flush()` per record. 101 write calls, 1.1 ms of wall time [S52].

Your prediction

Add one `os.fsync()` per record — the third arrow on the stack you drew, level 3 to level 4.

How long?

Write a number in milliseconds. Not a factor, a number.

[S52] [demo/durability/README.md](https://on-drasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md) — four-mode counts, 2026-09-20. <https://on-drasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md>

One more prediction, and this one is quick.

I have run the same program four ways. One of those ways flushes after every record. That pushes each record out of the Python buffer with a write system call. A hundred records, a hundred and one write calls, one point one milliseconds of wall time [S52].

Now the question. I change one line. After each flush I also call `os.fsync`. That is the third arrow on the diagram you drew this morning, from the page cache down to the device. It asks the kernel to push that record to the device before returning.

Same hundred records. Same hundred and one write calls. The only difference is a hundred `fsync` calls on top.

How long? Give me a number in milliseconds, not a multiplier. I want you to commit to a magnitude.

Ten seconds. Write it.

Hands up for under five milliseconds. Under twenty. Under a hundred.

51 51. Observe: sixty times, for the same hundred writes

On screen:

Part 6 · Durability

MODE	WRITE CALLS	FSYNC CALLS	WALL TIME
<i>default</i>	6	0	0.2 ms
<i>flush per record</i>	101	0	1.1 ms
<i>fsync per record</i>	101	100	67.3 ms
<i>unbuffered</i>	101	0	1.0 ms

Sixty times the wall time, for the same hundred write calls plus a hundred fsync calls [S52]. The cost is the waiting, not the extra crossings [inference].

Microsoft concedes the same: FlushFileBuffers "can be inefficient when used after every write to a disk drive device" [S40]. Most programs never pay it [speculative].

[S52] demo/durability/README.md — four-mode counts, 2026-09-20. <https://on-drasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md> [S40] FlushFileBuffers function (fileapi.h) — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-flushfilebuffers>

Sixty-seven milliseconds [S52]. Against one point one.

Read the table across. The write count is identical in three of the four rows. A hundred and one calls. What changed is a hundred extra calls, and each one waits for the device.

And notice where the cost is not. A hundred extra crossings did not cost sixty-six milliseconds. Lecture 2 measured a crossing in microseconds. The cost is the waiting, not the crossing.

And this is not a Linux quirk. Microsoft's own documentation for the Windows equivalent concedes the same thing about the same pattern. FlushFileBuffers can be inefficient when used after every write [S40]. That is the vendor writing about its own function.

Here is my reading of that gap, and it is only my reading. A text editor does not pay it on every keystroke. A database does. I have measured neither program [speculative].

One number to notice in passing, and I will come back to it. The tracer reported two point two milliseconds inside fsync against sixty-seven milliseconds of wall time. The other sixty-five were the process waiting for the device. Count under the tracer, time without it. Lecture 2 told you that and here is the instance.

52 52. What `fsync()` promises, and what it does not

On screen:

*Part 6 · Durability**It crosses one boundary. The next is not yours to check.**1**Does**Transfers the file's data "to the disk device", "including ... flushing a disk cache" [S67].**2**Waits for**It "blocks until the device reports that the transfer has completed" [S67].**3**Cannot check**Whether the device flushed. No trace tells "an honest flush from a fast lie" [V5].**Durability is a sequence of flushes, each crossing one boundary, each able to fail alone [V44]. The device reports.**[S67] fsync(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/fsync.2.html>*

Now the call itself, in the manual's own words.

Fsync transfers the file's modified data to the disk device [S67]. The page says this includes writing through or flushing a disk cache, if the device has one [S67]. Good. That is level three to level four, and it is what you want.

It blocks until the device reports that the transfer has completed [S67]. Also good. It waits.

Read the last four words again. The device reports. The whole guarantee rests on that report being accurate, and last week I told you that no software in this course can check it [V5].

The sentence at the bottom is from my own note, and I want you to keep it [V44]. Durability is not a property of a write. It is a property of a sequence of flushes. Each flush crosses exactly one boundary. Each one can fail on its own.

And this is not a theoretical worry about the last step. SQLite's documentation records that "often the IDE disk control lies and says that data has reached oxide while it is still held only in the volatile control cache" [V86].

So fsync is not the fix. Fsync is one crossing in a chain, and it is the last one you can see.

53 53. Windows: the same shape, and now measured

On screen:

Part 6 · Durability

The Windows twin of the four-mode run, on my Windows 11 VM [S52].

IOCost .EXE , 100 × 4 KiB	KERNEL WRITE OPS	WALL
<i>WriteFile into the cache</i>	100	0.7 ms
<i>FlushFileBuffers per record</i>	100	27.9 ms
<i>FILE_FLAG_WRITE_THROUGH</i>	100	25.2 ms
<i>FILE_FLAG_NO_BUFFERING</i>	100	9.9 ms

Forty times, for asking the device to confirm each record [S52]. Linux's figure was sixty. The two ratios are not comparable: these records are 4 KiB, the Linux ones were 64 B [S52].

Now read the count column: 100 in every row.

[S52] demo/durability/README.md, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md> [S40] FlushFileBuffers — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-flushfilebuffers>

Some of you are on Windows, so this is a measurement and not a paragraph of documentation. The program is iocost.exe. It is the twin of save.py: a hundred records, four modes, timed with QueryPerformanceCounter and counted with GetProcessIoCounters, which is Lecture 2's counter [S52].

Flushing after every record cost twenty-seven point nine milliseconds against nought point seven [S52]. Forty times. On Linux the ratio was sixty. Do not divide one by the other. These records are four kibibytes and the Linux records were sixty-four bytes [S52]. The shape is the same on both platforms. The two ratios are not comparable [inference]. Microsoft's own page concedes it about its own function [S40].

Now the column I want you to argue with. The kernel counted a hundred write operations in all four modes, including the two that go straight past the cache. For two lectures I have told you to count crossings. Here the count is the one number that says nothing, and the wall time is the number that does. The count says how many times the program asked. The time says who waited [inference].

I traced it too, filtered to three calls. Flush mode: a hundred and one NtWriteFile and a hundred NtFlushBuffersFile. Default mode: the same writes and not one flush [S52]. The hundred and first write is the done line going to the console.

Three limits. The record sizes differ from the Linux run, as the slide says [S52]. This is an x64 binary under emulation on an ARM64 virtual machine [S52]. Read the shape, not the milliseconds. And under the tracer the nought point seven became twenty-six point six [S52]. Count traced, time untraced, on Windows too.

54 54. The two kernels write in opposite orders

On screen:

Part 6 · Durability

The same watch, on Windows, until the counter returned to baseline [S52].

WRITTEN	WINDOWS CACHE MAN- AGER	LINUX KERNEL
4 MiB, three runs	0.5 s · 1.0 s · 0.5 s	5.0 s · 5.5 s · 5.5 s
40 MiB, one run	14.1 s	0.5 s

*Windows' **lazy writer** had the small file out inside a second. Linux waited for its flusher's five-second wake. On the large file they swap — Linux wrote it at once, Windows trickled 512 pages a pass [S52].*

Both clocks are a kernel's. Neither is your program's [inference].

[S52] demo/durability/README.md, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md>

Same demonstration, same four options on your page, other platform. There is no proc meminfo on Windows. So watch_dirty_win32.py samples the Cache Manager's own performance counters instead: Cache Dirty Pages, and Lazy Write Flushes per second [S52].

Read the table across and notice that your prediction was right on both platforms, for different files. Under a second was the Windows answer for four mebibytes. About five seconds was the Linux answer for the same file. Then the forty-mebibyte run swaps them over.

So do not carry away that one kernel is quick and the other slow. Carry away that each one has its own rules, and that neither rule is in your program.

One honest note on the big run, the same shape as the Linux honesty slide. I can see that the pages went out in steady passes of about five hundred and twelve pages. They came roughly every second and a half. Then a final burst finished the file. Which Cache Manager rule produced that pattern is my reading of the timeline, not something this demonstration shows [inference]. Windows Internals is the book to read, and I am not going to paraphrase it into a fact.

What I can say is what you watched. The program said done, and fourteen seconds later the kernel agreed.

55 55. Predict: your `fsync()` failed — what does the retry return?

On screen:

Part 6 · Durability

The device rejected the write, so `fsync()` returned `-1` with `EIO`. The obvious repair: call it again.

1

The second call

`-1` again · 0, bytes on the device · 0, bytes gone.

2

Who found out

A database team, in 2018, in production [S14].

One of three, written down next to your Part 1 prediction [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/> · [S41] `fsyncgate`. <https://danluu.com/fsyncgate/>

Part 5 gave you a rule with no story behind it: never retry a failed close [S4]. Here is the story, one level down, and it starts with a prediction.

Picture the situation. Your program called `fsync`. The device reported a failure, so `fsync` returned minus one with `EIO`. Every instinct you have says try again. Transient errors are what retry loops are for.

Three options for what the second call returns. Minus one again, which is what a retry loop expects. Zero, with the bytes safely on the device, which is what a retry loop hopes for. Or zero, with the bytes gone.

Write one down. In a moment you see the trace, produced by the engineer who reproduced this after a database lost data in production [S41].

56 56. Observe: the retry returns success, over data that was never written

On screen:

Part 6 · Durability

2018, and the case is known as `fsync-gate`. A PostgreSQL developer found that retrying a failed `fsync()` "is not OK at least on Linux" [S41].

[code]

That is a real reproduction against an error-injecting device. Its author annotated the last line "`fsync on file with still-dirty data+EIO returns success BAD`", for a byte that "certainly never got written" [V53].

[S41] `fsyncgate` — the `pgsql-hackers` thread, archived. <https://danluu.com/fsyncgate/>

Read the trace. It is the same shape of output you read one line of last week [V53].

Line one. `Fsync` fails and reports an I O error. Correct behaviour. Something went wrong with the device. And note the boundary the person who found this drew: retrying is not OK "at least on Linux" [S41]. He did not claim it for every system, and two slides on you will see why that matters.

Line four. The program calls again, the way any of us would, and gets zero. Success. Over data that was never written to anything.

The person who reproduced this annotated that line himself. `Fsync on file with still dirty data returns success. Bad` [V53].

Check your prediction. If you wrote "minus one again", the kernel disagreed with you. If you wrote "zero and the bytes are on the device", so did the byte.

57 57. Explain: the retry does not retry

On screen:

Part 6 · Durability

Mechanical, not sloppy code: the retry does not re-attempt the failed I/O [V54].

1. **Reported once.** *The kernel marks the page failed, reports it to the first `fsync()`, and clears the mark [S41].*
2. **Already gone.** *A failed writeback drops the data and marks the page clean, so a retry has nothing to write [V54].*
3. **So the retry lies.** *"It reads a status register that the previous read destroyed, over a page cache the data has already left" [V89].*

The result: "how PostgreSQL lost data silently for twenty years" [V90].

Now why. And the reason is not that somebody wrote sloppy code. The retry does not re-attempt the I/O at all [V54].

Step one. The kernel tried to write the page, the device said no, and the kernel marked the page as failed. The first `fsync` call collected that mark and reported the error. Collecting it cleared it [S41].

Step two. The kernel also dropped the page. A failed writeback discards the data and marks the page clean, because the kernel will not keep unwritable memory around forever [V54]. So the bytes left the page cache when the first call failed.

Step three. The second `fsync` finds a clean page and no error mark. It returns zero. It is telling the truth about the state it can see, and that state has no memory of the failure. "It reads a status register that the previous read destroyed, over a page cache the data has already left" [V89].

So the obvious repair makes the failure silent. A failure that is both silent and cleared by the retry is the hardest kind to find. This one is "how PostgreSQL lost data silently for twenty years" [V90].

Notice the shape. In Part 5 you saw the slide "Never retry a failed close". This is the same rule, one level further down. Next: what the databases did about it.

58 58. What the engines did instead

On screen:

Part 6 · Durability

Three storage engines, three architectures, one answer, in one year.

PostgreSQL

[V40]

Retry replaced by a crash [V93].

2018

InnoDB

[V84]

Retry loop became a fatal error.

2018

WiredTiger

[V24]

No retry. Panic on a failed flush.

2018

A stopgap that "has now lasted eight years" [V56].

So what do you do when a call can report success over lost data?

Three shipped storage engines answered in 2018, and they answered the same way [V40]. You crash.

PostgreSQL replaced the retry with a panic and back-patched it to every supported release [V93]. InnoDB deleted a sleep-and-retry loop and substituted a fatal error [V84]. WiredTiger converted its sync paths to panic, and wrote the reason into the source comment [V24].

The argument is worth a sentence, because crash on error sounds like giving up. It is not. A database writes every change to a log first, then later writes the data files and deletes the old log. That second step is a checkpoint, and it is the moment the database throws away its own last copy. The fsync return code is the only evidence that lets it do that. Once that evidence may be false, the safe move is to discard the in-memory model and rebuild it from the log. Crashing is what preserves the log.

Two honesty notes. These are not three independent discoveries. WiredTiger's comment quotes the same LWN report the other two were reading [V24].

And this was a stopgap. My reading note says it has now lasted eight years [V56], which is a polite way of saying nobody found anything better.

59 59. Windows here is not "the same", and not "safe"

On screen:

Part 6 · Durability

It is undocumented, which is a third answer and a worse one.

•

What is missing

[V47]

The PostgreSQL wiki's matrix of post-fsync-failure behaviour has no Windows row at all.

•

Why nobody can settle it

[V55]

The Linux contract was inspectable, so it was discoverable. The Windows contract "cannot be settled by reading" [V55].

Linux documents a hostile contract: one error report, then the data is gone. That is actionable. "For a system that must not lose data, an unspecified contract is the worse of the two positions" [V91].

Now the Windows half of fsync-gate, and it is the clearest case I have of reading what a specification does not say.

Eight years after this was found on Linux, the PostgreSQL project maintains a table of how each operating system behaves after a failed fsync. Linux, four kernel ranges. FreeBSD. NetBSD. macOS. Illumos. DragonflyBSD.

There is no Windows row [V47]. Not a row saying unknown. No row.

Why has nobody filled it in? On Linux the logic lives in shared kernel code that anyone can read. On Windows, FlushFileBuffers is a thin wrapper that dispatches to whichever filesystem driver owns the volume [V83]. The answer can therefore differ per driver, and my note says the Windows contract cannot be settled by reading [V55].

So the correct sentence for an exam is neither "Windows is safe" nor "Windows has the same bug". It is: Linux documents a hostile contract, and the Windows one has never been settled [V55].

And for a system that must not lose data, an unspecified contract is the worse of the two [V91]. Learning to read what a specification does not say is a professional skill, and this is the clearest case I have.

60 60. Sync the directory too

On screen:

Part 6 · Durability

This part is yours, and the classic recipe is incomplete [V51].

*A file is two pieces of data in two places: its **bytes**, and its **name**, which lives in the directory. `fsync()` on the file flushes the bytes. The manual: "Calling `fsync()` does not necessarily ensure that the entry in the directory containing the file has also reached disk" [S67].*

So "create-write-fsync-close on a new file is the classic incomplete recipe" [V51]: after a crash the bytes are safe and the file may have no name.

The fix costs one line: open the directory, and `fsync()` that descriptor too [S67].

I have spent five slides on things you cannot control. Here is the part you can, and it is the part real code gets wrong.

Think of a file as two things. The bytes, and the name. They are stored in different places. The bytes belong to the file. The name is an entry in another file, the directory. When you call `fsync` on your file, you flush the bytes. The directory has not been touched by that call. The manual page says so plainly, and almost nobody reads that far [S67].

So the recipe everybody writes, create the file, write, `fsync`, close, is incomplete [V51]. After a crash you can have a file whose bytes are all there and whose name never made it. It is unfindable.

The fix is one more call. Open the directory itself, you get a descriptor, and fsync that too [S67]. Now the name is on the device as well.

61 61. Write-then-rename, and why it needs a sync

On screen:

Part 6 · Durability

How every editor saves, and how ext4 made it lose files in 2009 [S44].

[code]

Why the pattern. `rename()` swaps the name atomically: "there is no point at which another process attempting to access newpath will find it missing" [S70].

The trap. The rename is a small piece of metadata. The temporary file's bytes are still dirty in the page cache. ext4 wrote the rename first, so a crash meant "that existing file gets replaced with a zero-length file" [S44].

The fix. `fsync()` the temporary file before the rename [S44]. Then the name can only point at bytes already on the device [inference].

Third, and this is the one that hit real users. Write a temporary file, close it, rename it over the original. That is how every editor saves, and there is a good reason for it.

The reason is rename. Rename replaces the name in one step. The manual promises there is no moment at which somebody opening the file finds it missing [S70]. So a reader gets either the whole old file or the whole new file. Never a half-written one. That is what the pattern buys you.

Now the trap. Rename changes a directory entry. That is a tiny piece of metadata, and filesystems write metadata eagerly. The bytes of the temporary file are a different matter. They are sitting in the page cache, level three on your drawing, waiting for the writeback you measured in this part. ext4 in 2009 wrote the rename to disk before the bytes. If the machine crashed in between, the original file was already replaced, by a file with a name and no bytes. Ts'o wrote that up himself, about a filesystem he maintains [S44].

The fix is the same call as before. fsync the temporary file, then rename [S44]. After that, the name can only point at bytes that already reached the device.

So the summary of this whole part is not give up [inference]. The device you cannot check. The directory you can, and it costs you one line.

62 62. The respectable opposite view

On screen:

Part 6 · Durability

Most programs are right not to pay, until the failure is "silent" [V52].

1

Most need not pay

SQLite's own write-ahead-log setting "does lose durability", yet "a good choice for most applications" [S43].

2

Trusting the layer

Trusting the layer below is "a defensible ... position", and "the implicit default" [V52].

3

Blame the API

"no one ever, ever wrote creat(); write(); close(); rename(); and hoped they would get an empty file" [S45].

[S43] SQLite. https://www.sqlite.org/pragma.html#pragma_synchronous [S45] Aurora, LWN (2009). <https://lwn.net/Articles/351422/>

I would be teaching you badly if I left you thinking every program must call fsync. Most should not.

SQLite is the cleanest example, because it documents the trade instead of hiding it. Its recommended mode in write-ahead logging does lose durability, and it says so in those words. It still recommends the setting for most applications [S43]. A transaction can roll back after a power cut. For a phone address book that is the right trade.

My own note concedes the general position. Trusting the abstraction and covering the gap operationally is "a defensible engineering position for a great many systems", and it is "the implicit default" [V52].

The strongest version of the objection is Valerie Aurora's, and she is arguing against the position I have been teaching. Nobody ever wrote that four-call sequence hoping for an empty file [S45]. On her reading, a system that demands this knowledge from every programmer has already failed.

So where does the defensible position stop being defensible? My note names it exactly. It fails where the failure is silent [V52]. Backups do not help you if nothing told you anything was lost.

That is the line. Not "always fsync". Know which done you got, and know whether losing it would be loud or quiet.

63 63. Log first — and what if the log write fails?

On screen:

Part 6 · Durability

The write-ahead log protects you because of an order, and the order covers its own failure [inference].

1. **Order.** Log first, `fsync` ed; only then committed [V85]. The data file follows later, at a checkpoint [S71].
2. **Log `fsync` fails:** nothing was promised yet. PostgreSQL panics here too: "we already did the same for WAL data" [S71].
3. **Data-file `fsync` fails later:** "the only remaining copy of the data is in the WAL" [S71]. Crash, replay.

The log wins only because it is made durable first, one `fsync` per commit — the 67 ms you saw [S52].

A fair question, and one you should ask of every log-based design: the log only helps if it wins over the file. So what if the log write fails and the file write does not?

The answer is an order, and the order is the whole trick. Nothing is written to the data file until the same change is in the log and the log has been fsynced. And nothing is told to the client until that fsync returns. Committed means the log is on the device [V85].

Now run the failure through that order. The log's fsync fails. At that moment no client has been told committed. PostgreSQL treats that exactly like the data-file case, it panics, and the 2018 commit says so in one sentence: they already did the same for WAL data [S71]. When it comes back up, the transaction simply never happened, and no one was promised otherwise.

The other way round, the data file fails later, at a checkpoint. The log still has the change. The commit message says it: the only remaining copy of the data is in the WAL [S71]. Crash, replay, done.

So the log is not magically more durable than the file. It is made durable first, and paid for with one fsync per commit. That is the sixty-seven milliseconds you measured earlier in this part [S52]. Databases pay it. Your editor does not, and Part 6 is the reason [inference].

64 64. So when is it safe?

On screen:

Part 6 · Durability

The answer, and what it does not cover.

In software

The last done you see is fsync(), resting on the device's report [S67].

In hardware

A tested UPS keeps the host up. Level 4 is inside the drive, and needs its own power-loss protection [inference].

Neither covers

fsync-gate lost data with the power on [V53]. No battery helps [inference].

So also

Log every change first: "recover from the WAL after any failure" [V85].

SQLite trusts neither call — "neither ... works as advertised" [V49].

So. When is it safe.

In software, never completely. Every done in the stack is a handoff to the next layer. The last handoff you can watch is fsync, and fsync trusts the device [S67] [V5].

In hardware, the honest answer is the one I gave you last week without a slide. The only real answer to durability, and it is still best effort, is a well-tested uninterruptible power supply. Keep the power on long enough for the last handoff to finish.

I owe you one correction to my own sentence, and it comes from the diagram you drew this morning. A UPS keeps the host running. Level four is the cache inside the drive. A drive can lose power while the host still has some. So a battery upstairs does not defend it [inference]. What defends level four is capacitors in the drive — power-loss protection

— which is the thing to check on a datasheet rather than assume. So the honest pair is a tested UPS above and a checked drive below. Durability is the conjunction of every crossing succeeding, and each crossing can fail on its own [V44].

Then the part neither of them covers. Fsync-gate lost data with the power on and the machine running [V53]. A battery is no defence against a failure the kernel reports once and cannot report again [V54].

So the full answer has two halves. Keep the power on. And write every change to a log before you write the data. PostgreSQL's own documentation puts it plainly: "the only way to avoid data loss is to recover from the WAL after any failure is reported" [V85].

SQLite says it most plainly of anyone. It calls both flush primitives and trusts neither, because it has "received reports that neither of these interfaces works as advertised on many systems" [V49]. Its correctness rests on journalling and checksums instead.

That is the state of the art. Not a better call. A design that survives the call being wrong.

65 65. Lecture 4 starts here

On screen:

Parts 7 to 9 · The announced tail

Parts 7, 8 and 9 are built in full. If the clock has gone, Lecture 4 opens on this slide and we jump to part 10.

Stop the clock for thirty seconds, because I owe you an explanation of how the rest of today works.

Parts one to six and part ten were the plan for this slot. If we are on time, we carry on from here through parts seven, eight and nine. If we are not, I skip to part ten now, and Lecture 4 opens on this exact slide.

I am not cutting anything either way. I have written all three parts, captured every demonstration in them, put them in your handout, and the containers are in the repository today.

I am telling you this rather than quietly running out of time, for the same reason I tell you what a measurement does not prove. You should always know which part of a plan you are in.

Parts seven and eight are also the two topics Lecture 2 named and deferred. So if we reach them, we close two loops at once. Part nine is the one topic today that Lecture 2 never mentioned.

66 66. What a pipe is

On screen:

Part 7 · Waiting

The thing we are about to wait on, a hundred times over.

"Pipes and FIFOs (also known as named pipes) provide a unidirectional interprocess communication channel. A pipe has a read end and a write end" [S59].

- *A pipe is a 64 KiB buffer inside the kernel [S59]. One process writes into one end, another reads from the other.*
- *Both ends are file descriptors, so `read()` and `write()` work on them unchanged [S59].*
- *"If a process attempts to read from an empty pipe, then `read(2)` will block until data is available" [S59]. That word, block, is the next slide.*

Before the prediction, two words you need, because the whole of Part 7 is built on them. First word: pipe.

A pipe is a buffer that lives inside the kernel. It has two ends. One process writes into the write end, another process reads from the read end, and the bytes pass through the kernel's buffer in between. On Linux that buffer is sixteen pages, sixty-four kibibytes [S59]. When you type `ls`, a vertical bar, `wc` in a shell, the shell creates a pipe and connects the two programs to its two ends.

Both ends are ordinary file descriptors. That is why `read` and `write` work on a pipe without any change in your program [S59]. Part 8 makes that the whole point of the lecture's second half.

And one sentence from the manual that sets up everything that follows. If a process attempts to read from an empty pipe, `read` will block until data is available [S59]. Block. That is the second word.

67 67. Blocking, non-blocking, and letting the kernel wait for you

On screen:

Part 7 · Waiting

Three ways to ask a pipe "is there anything to read?" [inference]

1. **Blocking.** `read()` on an empty pipe parks your thread until data arrives [S59].
Fine for one pipe, hopeless for a hundred.
2. **Non-blocking.** Set `O_NONBLOCK`; now `read()` returns at once, with `EAGAIN`, when there is nothing [S59]. Ask all hundred, again and again: polling.
3. **Let the kernel wait.** `select()` and `poll()` take a list of descriptors and sleep until one is "ready", meaning a read "without blocking" is possible [S47]. One sleeping call instead of thousands of asks. `epoll` is Linux's faster form of the same idea [S46].

Second word: blocking. And the two things people built to avoid it.

Blocking is the default. You call `read` on an empty pipe and the kernel parks your thread. It does not run again until data arrives [S59]. For one pipe that is perfect, you use no processor while you wait. For a hundred pipes it is a problem. You asked pipe number one first, so you are asleep on pipe number one while pipe fifty has data waiting.

The first way out is non-blocking mode. You set a flag on the descriptor, `O_NONBLOCK`, and `read` stops waiting. If there is nothing, it returns immediately with an error called `EAGAIN`, try again [S59]. Now you can ask all hundred pipes in a loop. And then loop again. That is polling, and Program A in a moment does exactly this.

The second way out is to let the kernel do the waiting. `select` and `poll` are calls that take a whole list of descriptors and sleep until at least one of them is ready. Ready has a precise meaning in the manual, a read on it would not block [S47]. One call, one sleep, instead of thousands of questions. `epoll` is Linux's newer and faster version of that idea, with the list kept inside the kernel [S46]. Program B uses it.

Hold those three. Ask and wait. Ask and return. Let the kernel wake you. Now the prediction.

68 68. Predict: a hundred pipes, and nobody writing

On screen:

Part 7 · Waiting

Two programs, two seconds, zero bytes of data. They differ only in how they wait [S14].

The setup

A hundred pipes. Nobody ever writes to them. Both programs wait two seconds, then exit.

Program A

Asks every pipe in turn, non-blocking, round after round.

Program B

Registers all hundred once, then makes one call that sleeps.

The question

How many system calls does each one make? About a hundred · a thousand · thirty thousand · millions.

Two numbers, on paper. Neither program transfers a single byte.

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

Part seven, and the tail starts here.

Two Python programs. Both create a hundred pipes that nobody ever writes to. Both wait two seconds. Both then exit having received nothing.

The only difference is how they wait.

Program A asks every pipe in turn. Is there anything? No. Next pipe. Is there anything? No. Round after round, for two seconds. The reads are non-blocking, so each one returns immediately with nothing.

Program B registers all hundred pipes with the kernel once. It then makes a single call. That call sleeps until something arrives or the two seconds are up.

How many system calls does each make? Pick from about a hundred, a thousand, thirty thousand, or millions. Two numbers.

I will say one thing to help you. No data ever arrives. Every call either of these programs makes buys the same information, which is that nothing has happened. Each model pays its cost while nothing is happening [V61].

Thirty seconds.

69 69. Where the question comes from

On screen:

Part 7 · Waiting

The cost of waiting, not of I/O.

Kegel, 1999

[S55]

"So hardware is no longer the bottleneck."

Named

Gammo et al.

[S49]

Plus 10,000 idle connections: "degrade by up to 79%".

Measured

*Up to 79% of `select / poll` throughput gone, only idle connections added [V61]. No answer made a `read` faster: each changed "**where the interest set lives**" [V62].*

Kegel's 1999 revision lacks the idleness clause everyone quotes [S55] [V63].

[S55] The C10K problem, 8 May 1999 — Kegel. <https://web.archive.org/web/19990508164301/http://www.kegel.com/c10k.html> [S49] Gammo et al. <https://www.kernel.org/doc/ols/2004/ols2004v1-pages-215-226.pdf>

Before we run it, where this question comes from.

In 1999 Dan Kegel named the C10K problem. Ten thousand simultaneous clients on one machine. And his opening line disposes of the explanation everyone reaches for first. Hardware is no longer the bottleneck. That sentence is in the earliest archived revision of the page, 8 May 1999 [S55].

So what was? Not moving bytes for the busy connections. Waiting on the idle ones.

Five years later a group at Waterloo proved it, and the design of their experiment is the reason I trust it [S49]. They held the request workload fixed. Same requests, same bytes, same server. Then they added ten thousand connections that do nothing at all.

Throughput fell by up to seventy-nine per cent [S49]. Nothing was added except idleness [V61].

And notice what none of the answers did. Epoll did not make a read faster. Every one of the answers changed where the interest set lives and who re-states it [V62].

Now a precision point, and it matters because you will find the Kegel page yourself. The famous sentence about most descriptors being idle is not in the 8 May 1999 revision [S55]. It appears in a later revision [S48]. My note calls the modern framing a sharpening of the original rather than a quotation of it [V63].

So cite the revision, or better, cite the measurement. The measurement is the thing that settles it.

70 70. A thread each, and why that stopped working

On screen:

Part 7 · Waiting

A hundred pipes, a hundred blocking threads — thread-per-connection — failed on two bills [S59].

1

Memory

Kegel priced the stacks: 2 MB each, "not an uncommon default value", against 1 GB of user-accessible address space. 512 threads, then no virtual memory left [V94].

2

Scheduler

Banga, Mogul and Druschel priced the rest: it "scales badly as the number of threads increases" [V95].

Either way the bill is per connection, not per byte [V96]. My note calls that "the same mistake at two altitudes" — my reading, not either source's [V96].

Someone is already thinking it. A hundred pipes, so start a hundred threads, each blocking on its own pipe.

That design has a name: thread-per-connection. It failed on two bills.

The first is memory. A thread needs a stack. Two megabytes each was, in Kegel's words, not an uncommon default. Give a 32-bit process one gigabyte of user address space and you run out at five hundred and twelve threads [V94].

The second is the scheduler. The costs of scheduling, context switching and thread-state storage all grow with the thread count [V95].

Both bills are charged per connection, not per byte [V96]. That reading is my note's own, not either source's [V96]. This is history and arithmetic, not a measurement I made today [inference].

71 71. Observe: two seconds, zero bytes, counted

On screen:

Part 7 · Waiting

[code]

28,017

read calls, 28,000 of them failing with EAGAIN [S53]

1

epoll_pwait call, which slept for the whole two seconds [S53]

EAGAIN is not a fault. It is the kernel answering "nothing yet" to a non-blocking read, and `strace -c` files it under `errors` [S53].

[S53] demo/waiting/README.md — captured 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/waiting/README.md>

Twenty-eight thousand and seventeen against one [S53].

Read the left column. Twenty-eight thousand reads, and twenty-eight thousand of them returned an error. Two seconds of work to learn that nothing happened.

Read the right column. A hundred epoll control calls, which are the registrations, paid once. Then one wait call.

One thing to notice about the percentage column, because it does not say what you expect. It is processor time inside the kernel, and it is independent of wall-clock time [S1]. The single wait call shows under two per cent because a sleeping call costs almost no processor time. It still occupied the whole two seconds.

Hands up who wrote thirty thousand for the first. Hands up who wrote about a hundred for the second.

Now one word you need, because it will mislead you. EAGAIN. The tracer prints it in a column headed errors, and it is not an error [S53]. It is the kernel answering a non-blocking read with nothing yet. The polling program is working exactly as written. Twenty-eight thousand correct answers to a question that had no news in it.

You will meet a strace summary with a big errors column. Check what the errors actually are first. Only then conclude that something is broken.

72 72. What those counts do and do not say

On screen:

Part 7 · Waiting

Three things about 28,017 and 1 before you carry them anywhere [S53].

- 1. **28,017 grows with time.** Twenty seconds of polling makes ten times the calls, and every one of them learns nothing [S53].*
- 2. **1 does not.** The `epoll` program paid a hundred registrations once, then one wait, however long it waits [S53].*
- 3. **28,017 is too low.** The tracer slows every call. Untraced, the same loop "asked 1.3 million times in one second" [S53].*

So: count under the tracer, time without it [S1]. Brendan Gregg measured a loop of nothing but system calls running 442× slower under `strace` [S66] [V17].

Three things about those numbers before you carry them anywhere.

First, the twenty-eight thousand is a rate, not a total. Run the program for twenty seconds instead of two and it makes ten times as many calls, and every one of them learns nothing [S53]. That is the shape of the cost: it grows with time, not with data.

Second, the `epoll` program's one wait call is a constant. Run it for twenty seconds, or twenty minutes, and it still makes one. The hundred registrations were paid once. Two different curves, and that is the entire content of part seven.

Third, and this is the uncomfortable one. Twenty-eight thousand is a floor, not the real rate. The tracer slowed the loop down. Without the tracer, the same laptop asked one point three million times in one second [S53]. So the rule you take into the assignment: count under the tracer, and time without it [S1]. Brendan Gregg's measurement of a loop that does nothing but system calls came out four hundred and forty-two times slower under strace [S66] [V17]. The tracer is not free, and it is least free exactly where you most want to count.

73 73. `epoll` is three calls

On screen:

Part 7 · Waiting

1. `epoll_create1` makes an **interest set** that lives in the kernel between calls [S46].
2. `epoll_ctl` adds or removes one descriptor. Our hundred registrations [S53].
3. `epoll_wait` sleeps until a registered descriptor is readable. Our one call [S53].

`select` and `poll` hand the kernel the whole list on every call [S46]. `epoll` states it once, which is why Program B's count was a constant.

Readable has a precise meaning: a read "without blocking" is possible, because at least one byte is waiting or the writer has closed its end [S47] [inference]. It does not say how many bytes.

You have just read a trace of an API I have not named yet. Here it is, because I want you to be able to say what the three calls are rather than only recognise the shape.

Call one creates the interest set. The set lives in the kernel, and you get a descriptor that names it. That is the structural move: the set survives between calls, so nobody has to re-state it [S46].

Call two adds one descriptor to that set. In our program it ran a hundred times, once per pipe, and then never again [S53]. Registration is paid once however long you then wait.

Call three sleeps. It returns when one of the registered descriptors is readable, or when the timeout expires. In our program it ran once and slept for two seconds [S53].

The difference from `select` and `poll` is where the list lives. Those two take the whole list as an argument, every call, and the kernel walks it every time [S46]. `epoll` keeps the list.

One more word, because it is used loosely. Readable. The manual defines ready precisely: an operation, here a read, could be performed without blocking [S47]. On a pipe that means at least one byte is there, or the writer has closed and a read would return end of file at once. Readable says nothing about how many bytes [inference]. That matters on the next slide.

74 74. Level-triggered and edge-triggered

On screen:

Part 7 · Waiting

One flag. Get it wrong and a connection goes quiet [inference].

Level-triggered — the default

Unread data is reported on every wait. The manual: "a faster poll(2)" [S46].

Edge-triggered — EPOLLET

Reports only when something new arrives [S46]. Read 1 kB of a 2 kB arrival and the next wait "will probably hang" [S46].

The rule for EPOLLET : every time you are woken, read everything — until EAGAIN — because the kernel will not mention the rest again [S46]. Non-blocking descriptors, or that loop never ends [S46].

One design choice inside epoll, because you will meet the flag and the wrong guess is expensive.

Level-triggered is the default and it is what our program used. While a descriptor still has unread data, every wait reports it again. The manual's own summary of that mode is reassuring. It calls level-triggered epoll a faster poll, usable wherever poll is used [S46].

Edge-triggered is the other mode, and the man page's own worked example is the bug in miniature. A writer puts two kibibytes in a pipe. Your wait reports the pipe. You read one kibibyte. You wait again, and it probably hangs, with the other kibibyte still sitting there [S46]. The mode reports changes, and nothing changed.

So the rule, and I want you to be able to say it in one sentence. In edge-triggered mode, every time you are woken, you read all of the unread data. You keep calling read until it says EAGAIN, nothing left. Only then do you wait again [S46]. And for that loop to end, the descriptor has to be non-blocking, otherwise the last read blocks instead of returning EAGAIN [S46]. Break the rule and nothing errors. The connection just goes quiet.

75 75. Three ways to wait, and we watch all three

On screen:

Part 7 · Waiting

One ladder. Each rung answers "who gets parked, and where" [V69].

1

Polling

The program asks each descriptor. Our 28,000 failed reads [S53].

2

Readiness

The kernel says "you may read now" [S46]. Our one epoll_pwait [S53].

3

Completion

The program hands over a buffer. The kernel says "I have read for you" [V74].

*In a trace: many failed reads means **polling**, one sleeping call means **readiness**, no reads at all during the wait means **completion** [inference].*

[S46] epoll(7) — Linux man-pages. <https://man7.org/linux/man-pages/man7/epoll.7.html>

Three ways, and both of our programs were on the first two rungs.

Polling. The program asks. Every descriptor, every round, for as long as it waits. That was twenty-eight thousand reads [S53].

Readiness. The program registers its interest once and then makes one call that sleeps. The kernel wakes it and says you may read now. The program still does the read itself [S46]. That was the epoll program.

Completion. The program hands the kernel a buffer and says read into this. The kernel does the read and reports that it is done. On Windows this is overlapped I O and I O completion ports, and it is the native model there. We run it later in this part, on Windows, traced [S53].

My note frames all three as one question rather than a feature list, and it is the best sentence I have for this. Who gets parked, and where [V69].

Now the practical part, and this is what I want you to be able to do. Look at a trace and tell me the model. Many failed reads means polling. One sleeping call means readiness. No reads at all during the wait, with a completion dequeue instead, means completion [inference]. You will see all three today.

And notice when each cost is paid. All of it is paid while nothing is happening.

76 76. Readiness is permission to try, not a promise

On screen:

Part 7 · Waiting

The word ready sounds like a guarantee. It is not one.

`select(2)` , BUGS: it "may report a socket file descriptor as "ready for reading", while nevertheless a subsequent read blocks" [S47].

1

So use non-blocking mode

It is "safer to use `O_NONBLOCK` on sockets when blocking is undesirable" [S47].

2

The model

"permission to try, not a promise of success" [V58].

Ready reports a moment that has passed, on `select` and on `epoll` alike [V92]. A done, one rung up [inference].

[S47] `select(2)` — Linux man-pages. <https://man7.org/linux/man-pages/man2/select.2.html>

One caveat that catches students and production code equally.

Ready does not mean the read will succeed. The manual page for select says so about itself, in its own bugs section. It may report a socket as ready for reading, and a later read still blocks [S47].

How? Data arrives, the kernel marks the descriptor ready, and then the data is discarded for a checksum error. The report was correct about a moment that has passed.

The consequence is a rule you follow without thinking about it. Put the descriptor in non-blocking mode even in code that has just been told it is ready. The page recommends exactly that, calling it safer when blocking is undesirable [S47]. The same holds for epoll [V92].

My note has the sentence I want you to keep. A readiness notification is permission to try, not a promise of success [V58].

And notice what that is. It is today's thesis again, one rung up the ladder [inference]. The kernel said done about its own state at the instant of the call. You read it as done about your next call.

Same word. Different subject. Third time today.

77 77. The real difference is who owns the buffer

On screen:

Part 7 · Waiting

Not the API shape. The ownership, and it is why porting between the models is a re-write.

	READINESS	COMPLETION
<i>when you supply the buffer</i>	<i>after you learn the descriptor is ready [V75]</i>	<i>at submission, before any data exists [V74]</i>
<i>who owns it while waiting</i>	<i>you — the kernel holds no pointer [V75]</i>	<i>the kernel — you must not touch or free it [V74]</i>
<i>one buffer can</i>	<i>be shared across ten thousand connections [V75]</i>	<i>serve one outstanding operation [V74]</i>

My note calls ownership "the load-bearing row" [V57]. Microsoft's contract: clients "agree not to disturb the buffers in any way until the network has consumed the buffer's contents" [V74].

If you remember one structural fact from part seven, make it this one.

The two models differ in where the copy happens relative to the notification. Everything else follows from that.

In a readiness model, you supply the buffer after the kernel tells you data is waiting. The kernel holds no pointer into your address space between notifications, so one scratch buffer may be shared across ten thousand connections [V75].

In a completion model, the buffer must exist before the data does [V74]. You need one buffer per outstanding operation [V74]. It belongs to the kernel until the completion packet arrives [V74]. Microsoft's own wording is that clients agree not to disturb the buffers until the network has consumed the contents [V74].

My note calls this the load-bearing row [V57], and I agree with it. Count the consequences. One buffer per outstanding operation, held for the whole wait. Multiply by ten thousand idle connections and you have a memory bill the readiness model does not have.

What completion buys is a genuinely zero-copy path and many reads in flight on one socket.

This is why you cannot port an event loop between the models by changing function names. My note puts it exactly: code written for one model and mechanically ported to the other "is not slow — it is wrong" [V76].

78 78. It is not a Linux/Windows split

On screen:

Part 7 · Waiting

I have used the platforms as labels. Here is the correction.

1

The honest statement

"the readiness/completion split is not a Linux/Windows split" [V59].

2

The control case

BSD and macOS use kqueue, which is readiness [V81]. Not a Linux quirk [V59].

3

And both have crossed

Linux built a completion interface, Windows built a shared-memory ring [V68].

The useful question is not which system a program runs on. It is which done it is handed [inference].

I have been saying readiness on Linux and completion on Windows, and I need to correct myself before you write it in an exam.

My own note says it plainly. The readiness and completion split is not a Linux and Windows split [V59]. It is a split in the design space, and the two platforms entered it from different sides.

The control case is BSD and macOS. They classify with epoll: readiness notification, and the application does the I O [V81]. So readiness is not a Linux quirk. It is the Unix family's answer [V59].

And both platforms have since crossed. Linux, which had readiness for thirty years, built a completion interface. Windows has had completion since Windows NT. It built a shared-memory ring instead [V68]. That is a different thing, and I come back to it at the end of this part.

So what should you actually say? Not "Linux does readiness". Say that each platform's native model is the one its libraries and its documentation assume, and that both models exist on both systems now.

The useful question about a program is not which system it runs on [inference]. It is which done it is handed. That is the question we started the hour with.

79 79. The trap this demonstration sets

On screen:

Part 7 · Waiting

We measured pipes. A regular file behaves differently, and the difference is silent.

•

Readiness and regular files

[V73]

epoll "has nothing useful to say about a regular file, which is always "ready"". The call returns at once and you learn nothing.

-

Why a completion model exists at all

[V88]

Linux "grew a completion interface because readiness could not express what its users needed".

So when you do meet these interfaces, do not expect a disk file in an `epoll` set to wait. It will not [inference].

One warning, because it is the first thing that catches people when they do write one of these loops.

We demonstrated readiness on pipes. Pipes are a good teaching case because a pipe with no writer genuinely has nothing to give you, and the kernel can say so.

A regular file on disk is different. `Epoll` has nothing useful to say about it, because a regular file is always ready [V73]. Ask whether you may read from a disk file and the answer is yes, immediately, every time, whether or not the data is in memory.

The failure is silent. Your loop does not error. It spins, and it looks like your code is wrong.

And that gap is not a curiosity. Linux grew a completion interface because readiness could not express what its users needed [V88]. The regular file is the case it could not express [V73].

So the sentence to write down is this. Readiness works on things that can be not ready [inference]. Sockets, pipes, terminals. Not files.

80 80. Four Windows words before the Windows pair

On screen:

Part 7 · Waiting

The same two programs, in Windows vocabulary [S53].

1. **Named pipe.** *"a named, one-way or duplex pipe for communication between the pipe server and one or more pipe clients" [S72]. A pipe with a name, openable by unrelated processes.*
2. **PeekNamedPipe .** *Looks without reading: it copies data "without removing it from the pipe" and "always returns immediately" [S73].*

3. **Overlapped ReadFile** . A read that returns before the data exists; the kernel fills your buffer later [V74].
4. **Completion port, GetQueuedCompletionStatus** . A queue of finished operations; the call takes one, and "If there is no completion packet queued, the function waits" [S74].

Four words, and then the Windows pair reads like the Linux pair.

A named pipe is the Windows pipe you will actually meet. Microsoft defines it as a named, one-way or duplex pipe between a pipe server and one or more clients [S72]. The name is the point: two unrelated processes can both open it by name, where a Linux pipe has to be inherited from a parent. Ours are local and nobody ever writes into them.

PeekNamedPipe is the polling call. It looks into the pipe without taking anything out, and the documentation says it always returns immediately, even when the pipe is empty [S73]. That is the exact counterpart of the non-blocking read on the last slide.

Overlapped ReadFile is a read that does not wait for the data. You give the kernel a buffer and a place to report back, and the call returns at once. The kernel fills the buffer when data arrives, later, while your program does other things. My note puts it precisely: the buffer must exist before the data does, and it belongs to the kernel until the completion arrives [V74].

The completion port is where the kernel reports back. It is a queue of finished operations. GetQueuedCompletionStatus takes one entry off that queue, and if the queue is empty, it waits [S74]. That is the one sleeping call, the twin of `epoll_wait`.

81 81. Predict: the same hundred pipes, on Windows

On screen:

Part 7 · Waiting

Completion is the rung we have not yet watched run [S14].

Program C

A hundred named pipes. PeekNamedPipe on each, round after round — the Windows polling twin.

Program D

One overlapped ReadFile posted per pipe, all bound to one completion port, then one GetQueuedCompletionStatus that sleeps.

The question

Under NtTrace, how many native calls does each make in two seconds — and how many reads does Program D do while it waits?

Two numbers again. Both programs are in `demo/waiting/` [S53].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/> [S53] `demo/waiting/README.md`, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/waiting/README.md>

Part two set a standard: one program, both platforms, traced on the same day. I am holding part seven to it, so here is the Windows pair.

Program C is the polling twin. A hundred named pipes with clients connected that never write, and PeekNamedPipe on each one, round after round.

Program D is the completion program, and read its description carefully because it is the rung we have only described. It posts one overlapped ReadFile per pipe, up front, before any data exists. It binds all hundred pipes to one completion port. Then it makes one call, GetQueuedCompletionStatus, and sleeps.

Notice what is strange about that. The reads were already submitted. The kernel is holding a hundred of your buffers while nothing happens. That is what handing over ownership means, and it is the row the ownership table called load-bearing [V57].

So: two numbers. How many native calls does each program make in two seconds. And the second one is the question I actually care about — how many reads does the completion program do while nothing arrives?

Write both down. These are captured runs on my Windows eleven virtual machine, traced with NtTrace, and I am about to show you the counts [S53].

82 82. Observe: Windows, traced, both ways

On screen:

Part 7 · Waiting

[code]

3

native calls per PeekNamedPipe: create an event, control the pipe, close [S53]

1

NtRemoveIoCompletion, which slept the whole two seconds [S53]

*Untraced, the polling program made **330,100** peeks in two seconds. Traced, it made 2,500 [S53]. Count traced, time untraced.*

[S53] demo/waiting/README.md — NtTrace64 captures, Windows 11 build 26200, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/waiting/README.md>

Read the left column first. Two thousand five hundred peeks under the tracer. Every single one of them is three native calls, not one: create an event, do the pipe control, close the event [S53]. Seven and a half thousand crossings to learn nothing. The Win32 call you wrote is not the unit the kernel charges you for, which is exactly Lecture 2's lesson arriving in a new place.

Now the right column, and this is the shape I want you to be able to recognise. A hundred NtReadFile calls, all at the start, before any data exists. A hundred calls binding each pipe to the port. One call to create the port. Then one NtRemoveIoCompletion, which slept for two seconds and came back with STATUS_TIMEOUT [S53].

Count the reads made while waiting. Zero. The reads happened before the wait, and the kernel owns a hundred of your buffers for the whole two seconds.

That is the third tell from the ladder, made visible. Many failed asks is polling. One sleeping call is readiness. Reads submitted up front and then a completion dequeue is completion.

Two honesty notes. The hundred errors NtTrace reports on NtFsControlFile in both traces are the ConnectNamedPipe calls returning STATUS_PIPE_CONNECTED [S53]. That is an informational status, not a failure. And the PyWin32 pair shows the same shape one layer up, with the interpreter's own file reads mixed in [S53]. That is the reason to compare two runs rather than trust one.

83 83. Which numbers you may compare, and which you may not

On screen:

*Part 7 · Waiting**Two counts per platform. Compare within a platform, never across [S53].****You may, and shall, compare within one platform, both runs traced:***

- *Linux: 28,017 read calls against 1 epoll_pwait [S53].*
- *Windows: 8,153 native calls, three per peek, against 956, of which one NtRemoveIoCompletion slept the whole two seconds [S53].*

You may not compare: *28,017 with 330,100, since one is traced and one is not [S53]; or Linux with Windows, since they are different machines and read is not PeekNamedPipe [S53].*

What survives: the same shape, twice. Thousands of asks against one sleeping call, on two kernels that agree on little else [inference].

I have now put four large numbers on the screen in one part, and somebody is dividing the wrong two. So first, what you may compare, and what you shall compare in the assignment.

Within one platform, both runs traced the same way. On Linux, twenty-eight thousand reads against one epoll wait [S53]. On Windows, eight thousand native calls, which is two and a half thousand peeks at three native calls each, against nine hundred and fifty-six, and one of those slept the whole two seconds [S53]. Those pairs are honest. Same machine, same tracer, same clock.

Now what you may not do. Twenty-eight thousand and seventeen is a Linux count under strace. Three hundred and thirty thousand one hundred is a Windows count with no tracer at all [S53]. Those two do not sit on one axis. Nor do Linux and Windows against each other: a container on one host, a virtual machine on another, and a non-blocking read is not a PeekNamedPipe.

So what survives the refusal? The shape, within each platform. Thousands of asks against one sleeping call. Same answer, twice, from two kernels that disagree about almost everything else [inference]. Comparing across two measurements is the most common way a correct pair of numbers turns into a wrong claim [speculative]. You will do it in the assignment if I do not say so now.

84 84. What a ring is

On screen:

*Part 7 · Waiting**One word before the last slide of Part 7.**A ring is two queues in memory that your program and the kernel both see: "ring buffers which are shared between user space and kernel space" [S75]. "You place I/O requests you want to make on the SQ, while the kernel places the results of those operations on the CQ" [S75].**Why it matters here: every wait we counted today was a system call. With a ring you queue many requests and cross the boundary once [S75], or, with a kernel-side polling thread, "steady-state I/O submission needs **no syscalls at all**" [V11].*

One more word, because the last slide of this part uses it and I do not want it to sound like magic.

A ring is two queues, sitting in a piece of memory that both your program and the kernel can read and write [S75]. One queue is for requests. You write into it: read this, write that. The other queue is for results. The kernel writes into it: done, here is how many bytes [S75]. Neither side has to make a system call to put something in its queue, because the memory is shared.

Why does that matter after the hour you have just had? Because every wait we counted today was a system call. Twenty-eight thousand of them in one program, one in the other. With a ring you put many requests into the queue and cross into the kernel once to say go [S75]. And in one configuration, with a kernel thread watching the queue, you do not cross at all: my note says steady-state submission needs no syscalls at all [V11].

Keep the two ideas apart. The ring is how requests and results travel. Whether what travels is a readiness report or a completed operation is a separate question, and the next slide is about which platform took which [inference].

85 85. Where both platforms ended up

On screen:*Part 7 · Waiting**Not convergence. Each one took the half it was missing.**Linux**Readiness-native. Built a completion interface, `io_uring`, in kernel 5.1 [V79].*

Windows

Completion-native since NT [V80]. Built a ring, IoRing, in Windows 11 build 22000 [V80].

"Linux adopted completion semantics; Windows adopted the shared-memory transport" [V68]. The ring and the completion model are two separable ideas, and each platform took the one it lacked.

Learn IOCP, the I/O completion port, as current Windows practice. Rings are "the direction of travel", not the exam [V60].

Last slide of part seven, and it is the satisfying one.

Linux was the readiness platform. Then it built a completion interface called io_uring, in kernel five point one [V79].

Windows has had overlapped I/O and completion ports since NT, so it did not need a completion model. What it did not have was the shared-memory ring, and it built one, called IoRing, in Windows eleven build twenty-two thousand [V80].

Now read the sentence carefully, because the tempting summary is wrong. This is not two platforms converging on the same thing. Linux adopted completion semantics. Windows adopted the shared memory transport [V68]. The ring and the completion model are two separate ideas. They happened to ship together on Linux. Each platform took the half it was missing.

One instruction for this course, and it is in my own note. Teach I/O completion ports as current Windows practice, and mention the ring as the direction of travel [V60]. On Linux, epoll is the equivalent present tense [inference].

And one honest limit to close on. A ring moves the system-call boundary, and your tracer sits on that boundary. With a kernel-side polling thread, steady-state submission makes no system calls at all [V11]. The tracer sees the setup and then nothing. The thing you learned to watch last week is already moving.

86 86. Same mechanism, different clothes

On screen:

Part 8

A file, a pipe and a socket. One program, one trace, three destinations.

Part eight. Its Linux result is a two-line comparison, and its Windows result is the opposite of that.

Lecture 2 ended by naming networking and interprocess communication and saying they were the same mechanism. I asked you not to file them under something else. Today I show you the trace.

Three predictions and three runs. The first is a pipe. The second is a TCP socket, with netcat on both ends. The third is Windows, where the answer comes out the other way round [S57].

The program is the same iodemo you met in part three today. Nine lines of C. It contains no code about pipes and no code about networks.

87 87. Predict: which of the four calls changes?

On screen:

Part 8 · Three destinations

You know the file run by heart now. I change only the destination [S14].

[code]

I now run `./iodemo /dev/stdout | cat`. The bytes go into a pipe and `cat` reads them.

On paper

Which of the four calls changes — the `openat`, the `write`, the `close`, or nothing?

And

Does the byte count in the `write` change?

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

Here is the file run again. You have seen this trace three times today, so I am not going to read it.

Four calls. A failed open, a successful open, one write of fourteen bytes, a close.

Now I change one thing. The program takes its output path as an argument, and I give it slash dev slash stdout, and I pipe standard output into cat. So the same binary, with the same three `fwrite` calls, is now writing into a pipe.

Two questions. First, which of the four calls changes? The `openat`, the `write`, the `close`, or nothing at all.

Second, does the byte count in the `write` change?

Before you answer, think about which layer would have to change. The `fwrite` calls are in the C library. The `write` is in the kernel. Which of those two selects its behaviour from the destination? Remember that the library selects its buffering from the destination [V71].

Thirty seconds.

88 88. Observe: the file run and the pipe run

On screen:

Part 8 · Three destinations

[code]

The same four calls, the same fourteen bytes in one `write`, the same descriptor number [S54]. `cat` on the other end received one two three .

The `openat` argument changed. The call did not, the count did not, and the byte total did not [S54].

[S54] [demo/iodeemo-c/README.md](https://on-drasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/iodeemo-c/README.md) — the pipe run, captured 2026-09-20. <https://on-drasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/iodeemo-c/README.md>

Side by side. Read across the rows [S54].

Row one. Identical. The deliberate failure does not care.

Row two. This is the row where I owe some of you an apology. The call is still `openat` and it still returns descriptor three. But the argument is different, because I passed a different path. If you answered "the `openat` changes", you were looking at the argument, and you were right about what you saw.

So let me state the question more precisely than I did. Nothing changed about which calls were made, how many there were, or how many bytes each carried. The argument changed because I told it to.

Row three is the result. One `write`. Fourteen bytes. Descriptor three. Exactly what the file run did.

Row four. Identical.

And cat received one, two, three. The bytes arrived.

Nothing in the program depends on what is on the other end, and nothing in its trace records what was there.

89 89. The descriptor is the abstraction, not the file

On screen:

Part 8 · Three destinations

The reason is a kernel structure, not a convention.

"On Linux, anything that takes a file descriptor takes a socket, because the kernel routes both through the same dispatch table" [V64]. A pipe too [V29].

So `write(3, ...)` said done when the bytes were in the pipe. Had `cat` read them yet? Probably not — but I traced only the writer, so that is my reading [inference].

1

write call, for a file and a pipe alike [S54]

[S54] [demo/iodeemo-c/README.md](https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/iodeemo-c/README.md), 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/iodeemo-c/README.md>

Why does this work? Not by convention and not by luck.

My note on sockets puts it as a kernel structure. Anything that takes a file descriptor takes a socket, because the kernel routes both through the same dispatch table [V64]. A pipe is in that table too [V29].

So the descriptor is the abstraction. Not the file. The file is one of the things a descriptor can point at.

Now the thesis, one more time, in this new place. Write returned fourteen. What had happened at that moment? The bytes were in the pipe's buffer in kernel memory.

Had cat read them? Probably not yet. But I have to be careful here, and this is the kind of care I want you to copy. I traced the writer. I did not trace cat. So the claim that cat had not read them is my reading of how a pipe works, not something this trace shows [inference].

Same shape as part six. The call said done about a handoff. It said nothing about the other end.

90 90. Predict: the same program, writing to your terminal

On screen:

Part 8 · Three destinations

./iodemo /dev/stdout with nothing after it. No pipe, no file. The screen [S14].

*The same three fwrite calls: "one\n", "two\n", "three\n". The file run and the pipe run each made **one** write() of fourteen bytes [S54].*

On paper: how many write() calls when the destination is a terminal — one, or three? And a second answer: who decides, the kernel or the C library?

One more destination, and this time I want the prediction before you think you know the answer.

Same binary. Same three fwrite calls, one, two, three, each with its newline. You have seen it write to a file: one write of fourteen bytes. You have seen it write into a pipe: one write of fourteen bytes [S54].

Now I run it with nothing after it. Standard output is the terminal. The screen.

Two answers on paper. How many write system calls: one, or three? And who made that decision, the kernel or the C library? The second question is the one that matters, and the trace will answer it [S14].

91 91. Observe: three writes, not one

On screen:

Part 8 · Three destinations

[code]

Same binary, same container, run with a terminal attached (podman run -t); captured 2026-09-21 [S54]. Three system calls where the pipe run made one. The program did not change. The C library's decision did [inference].

Read it against the pipe run. Same `openat`, same descriptor three, same `close`. But three writes, four bytes, four bytes, six bytes, one per line [S54].

The only thing I changed is what is on the other side of descriptor three. Not the program, not the kernel, not the bytes. A terminal instead of a pipe.

So the program made three system calls where it made one before, and nothing in its source code says so. Something between `fwrite` and `write` decided differently. That something is the C library, and the next slide is its rule [inference].

92 92. Explain: the destination changed the trace, at the C library, not the kernel

On screen:

Part 8 · Three destinations

"Attached to a terminal, `stdout` is line-buffered... Attached to a pipe or a regular file, `stdout` is fully (block) buffered" [V71].

- 1. **The library asks once**, at the first write, what the descriptor is. A terminal gets line buffering: every `\n` becomes a `write()` [V71].*
- 2. **A file and a pipe** sit on the same side of that rule, so my first two runs agreed by construction [inference]: I chose two destinations the library treats the same.*
- 3. **The kernel's part never changed**: `openat`, `write`, `close`. Only how many times [inference].*

Try it yourself: `./iodemo /dev/stdout` in a terminal, then with `| cat` [S54].

Here is the rule, and it is the C library's rule, not the kernel's. Attached to a terminal, standard output is line-buffered. Attached to a pipe or a regular file, it is fully buffered, in blocks [V71].

The library asks the kernel once, at the first write, what kind of thing sits behind the descriptor. If it is a terminal, it switches to line buffering, and from then on every newline pushes a write out. Three lines, three writes [V71].

Now look back at the two runs I compared before this. A file and a pipe are both on the block-buffered side of that rule. So they agreed by construction [inference]. I did not discover that the destination makes no difference. I chose two destinations the library treats identically, and I owe you that honesty.

And notice what did not change. The kernel's calls are the same three names, on the same descriptor. The destination changed the trace at the library, above the boundary, in how many times the same call was made. Below the boundary, nothing knew [inference].

Try it yourself, both ways, one after the other [S54]. It takes thirty seconds and you will not forget it.

93 93. Predict: the network leg

On screen:

Part 8 · Three destinations

Same fourteen bytes, over a TCP socket, with netcat on both ends [S14].

The setup

./iodemo /dev/stdout | nc 127.0.0.1 9000, with nc -l listening. Both netcats traced.

The question

The listener receives the fourteen bytes from a TCP socket. Which call takes them off the wire?

Two options

read, the same call you would use for a file — or a socket-specific call.

One of two. Write it down.

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

Second leg, and this one goes over the network.

One honest thing about the setup first. Iodemo cannot open a socket by path. A socket is not reachable through slash dev slash stdout, and that is itself a fact about the boundary [V65]. So I use the pipe to carry the fourteen bytes to a netcat, and netcat puts them on the socket.

That means the traces of interest are netcat's, not iodemo's. I am tracing both ends.

The question. The listening netcat receives fourteen bytes from a TCP socket. Which system call takes them off the wire?

Option one. Read. The same call you would use on a file.

Option two. A socket-specific call. Something called `recv`, or `recvfrom`, or something else entirely.

If you have seen socket code in any language, it probably spelled the call `recv` rather than `read`. So the question is whether that spelling tells you anything about the system call underneath, or only about the library on top.

Fifteen seconds. One of two.

94 94. Observe: both netcats, traced

On screen:

Part 8 · Three destinations

[code]

The client puts them on the wire with `write(3, ...)`. The listener takes them off with `read(4, ...)`. The same two calls `iodemo` used for a file and for a pipe [S54].

What is new is only how the descriptor came to exist [S54].

[S54] [demo/iodemo-c/README.md](https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/iodemo-c/README.md) — the traced netcat pair, captured 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/iodemo-c/README.md>

Both ends, side by side [S54].

Left column, the client. It creates a socket. It connects. It reads our fourteen bytes out of the pipe with `read` on descriptor zero. Then look at the next line.

Write, descriptor three, fourteen bytes. That is the same call `iodemo` used for the file. That is the same call `iodemo` used for the pipe. Those bytes are now on a TCP connection.

Right column, the listener. Socket, bind, listen, accept. Four calls, and all four of them are about creating a descriptor. Then read on descriptor four, fourteen bytes.

Read. Not `recv`. Not `recvfrom`. The same call you would use on a file.

Hands up who wrote `read`.

So here is the shape of the whole part. Look at the first four calls in each column. Those are all different, and they are all about how the descriptor came into existence. After that line, everything is read and write [S54].

One detail in passing, and it connects the two halves of the tail. The client's connect returned EINPROGRESS. This netcat opened its socket non-blocking and waited for readiness before writing. That is part seven's mechanism, showing up in part eight's trace without being asked.

95 95. Then what are `send` and `recv` for?

On screen:

Part 8 · Three destinations

They are the program's choice of spelling, not the kernel's.

`send(2)` settles it in its own opening: "The only difference between `send()` and `write()` is the presence of flags. With a zero flags argument, `send()` is equivalent to `write(2)`" [S50].

1

Use them for a flag

`MSG_DONTWAIT` and `MSG_NOSIGNAL` have no `write()` equivalent [inference].

2

Otherwise it is style

They tell a human reader "this is a socket" [inference].

[S50] `send(2)` — Linux man-pages. <https://man7.org/linux/man-pages/man2/send.2.html>

So if read and write work, why does every socket tutorial you have ever read use send and recv?

The manual answers it in its first paragraph, and the answer is shorter than the question. The only difference between send and write is the presence of flags. With a zero flags argument, send is equivalent to write [S50].

That is not my paraphrase. That is the page.

So when do you want send? When you want a flag [inference]. There are several, and none of them has a write equivalent. Do not wait if there is no room. Do not raise a signal if the peer is gone. Those are real and you will use them.

And the rest of the time, the choice is style [inference]. Writing send in socket code tells a human reader that this descriptor is a socket. That is a legitimate reason, and it is a reason about readers, not about kernels.

What it is not is evidence of a different mechanism. Our netcat used read and write, and the bytes crossed a TCP connection.

96 96. The same interface is not the same contract

On screen:

Part 8 · Three destinations

Three boundaries on the result.

•

One send is not one receive

[V70]

TCP is a byte stream. Marking where a message ends is your job — "the single most common practical bug in student socket code".

•

A size effect

[V82]

"writes of less than PIPE_BUF bytes must be atomic" — 4 KiB here [V67] [V82].

•

The default is not total

[V87]

"a strong default, not a total function" [V65]. lseek on a socket means nothing.

Before you leave with "it is all the same", three ways it is not.

First, and this is the one I expect to cost you a night at some point [speculative]. One send is not one receive. TCP is a byte stream. You write fourteen bytes and the reader may get four then ten, or may get yours glued to the next message. Marking where one message ends is called framing. It is your job rather than the kernel's. My note calls it the single most common practical bug in student socket code [V70].

Which means our beautiful one-write-one-read result is partly a size effect. Fourteen bytes fits in anything.

Second, the same is true of the pipe. POSIX says writes below PIPE_BUF must be atomic [V67], and that threshold is four kibibytes on Linux [V82]. Above it, two writers' records can be shredded into each other [V82]. Our fourteen bytes were never going to show that.

Third. Everything is a file is a strong default, not a total function [V65]. Seek on a socket is meaningless, and the kernel marks sockets to skip the file-position locking they cannot use [V87].

So the accurate takeaway is the interface is shared and the contract behind it is not. The calls transfer. The assumptions do not.

97 97. Predict: the same three destinations, on Windows

On screen:

Part 8 · Three destinations

On Linux the destination changed nothing in the trace. Windows is the contrast [S57].

1

Pipe

WriteFile to an anonymous pipe. Fourteen bytes.

2

Socket

send() on a loopback TCP socket, fourteen bytes.

3

The same socket again

WriteFile on that socket, cast to a HANDLE.

*On paper: **is Windows the same as Linux here: only the destination changes, the call stays the same? Yes or no, and if no, which leg differs?** One program, three destinations [S57].*

[S57] demo/destinations-win32/README.md — captured 2026-09-20. <https://on-drasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/destinations-win32/README.md>

Some of you are on Windows, and I have just spent two demonstrations on a platform where the destination is invisible. Here is the other platform, and it answers differently.

One program, three legs, the same fourteen bytes each time. Leg one writes to an anonymous pipe with WriteFile. Leg two sends on a loopback TCP socket with send. Leg three writes to that same socket, this time by casting it to a handle and calling WriteFile on it [S57].

Your prediction is not which native call each leg makes, because most of you have never seen this API. It is whether the call stays the same when only the destination changes, as it did on Linux, and if not, which leg is the odd one out. And the interesting half is the comparison between leg one and leg three: same Win32 function, two different kinds of object. Does the kernel see one call or two?

Think about what you learned in part four before you answer. On Windows the stable boundary is the Win32 API, and the native layer underneath is Microsoft's business rather than yours. That cuts both ways here.

Thirty seconds. Both programs are in the folder, in C and in Python, and both were captured on the same virtual machine on the same day [S57].

98 98. Observe: the API changes the call, the object does not

On screen:

Part 8 · Three destinations

[code]

1 · pipe

[S57]

NtWriteFile, 14 bytes

file path

2 · send()

[S57]

NtDeviceIoControlFile, control code 0x1201f

device control

3 · WriteFile

[S57]

NtWriteFile — same handle as leg 2

file path

Legs 2 and 3 are one handle, two native calls. A plain WriteFile first failed with error 87 — Winsock sockets are **overlapped handles** [S57].

[S57] demo/destinations-win32/README.md, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/destinations-win32/README.md>

Read the handles, because the handles are the argument.

Leg one, the pipe, is NtWriteFile with length fourteen. Ordinary file write.

Leg two, the send on the socket, is not a write at all. It is NtDeviceIoControlFile with control code one two zero one f. That code is the AFD send code, and afd.sys is the driver behind Winsock. The decoding is from the sources, not from the tool: NtTrace has no AFD table and prints the number [S57].

Leg three is the one to stare at. Same handle as leg two — and NtWriteFile, the same native call as the pipe [S57]. So the file API reached the socket's file object directly.

Put those two facts together and you get the accurate sentence, which is not the one people usually say. On Windows the API you choose changes the native call. The kernel object underneath is a file object either way. Linux showed you the mirror image: one call, three destinations, nothing visible in the trace.

And one lesson from my first run, which failed. A plain WriteFile on the socket returned error eighty-seven, invalid parameter, because Winsock creates sockets as overlapped handles by default [S57]. Both programs now supply an OVERLAPPED structure and wait. Which means that on Windows the file API on a socket is part seven's completion model, whether you asked for it or not.

99 99. Windows: a SOCKET is not a HANDLE

On screen:

Part 8 · Three destinations

Third time today, and the same asymmetry each time.

*Linux**A socket is a file descriptor, and Linux makes the file-descriptor contract binding [V28].**Windows**An opaque SOCKET, and Windows makes Winsock the contract [V28].**The NT kernel does hold a file object underneath [V28] — which is exactly what leg 3 just wrote to [S57]. The type is not a C-runtime file descriptor, while the handle is a kernel file handle [V66].**So the platforms agree on the mechanism and disagree on which layer they promise [inference]. Third time today.*

Last content slide of part eight, and it is the third appearance of one idea.

On Linux a socket is a file descriptor and the file-descriptor contract is binding [V28].

On Windows it is an opaque SOCKET type and the binding contract is Winsock [V28].

Now the careful version, because the tempting summary is wrong and a curious student will catch you.

One layer down, Windows does treat a socket as a file. Underneath Winsock, the NT kernel is "also holding a file object" [V28]. You have just watched leg three write to that file object with NtWriteFile [S57]. But the contract Windows publishes is Winsock, not the file object [V28]. That is why leg three needed an OVERLAPPED structure to work at all. Do not write code that depends on the file object.

My note reconciles the two Microsoft pages that look like they contradict each other. The type is not a C runtime file descriptor. The handle is a kernel file handle [V66]. Both sentences are first-party and both are true, about different layers.

So the sentence for your exam is this one. The platforms agree on the mechanism and disagree on which layer they promise [inference].

You have now met that idea three times in one lecture. System calls in part four. Waiting in part seven. Sockets here. It holds at three different layers, which is why I keep returning to it.

100 100. Bytes you never touch

On screen:

Part 9

One call can carry several buffers. And the kernel can move a file your process never holds.

Part nine, and it is the one topic today that Lecture 2 never mentioned.

So far every done in this lecture has been about a place your bytes are. This part is about two calls that change how your bytes get there. You observe both of them best by what is missing from the trace.

Two demonstrations, two predictions, both in the container. The first is `writew` and it removes crossings. The second is `sendfile` and it removes copies. Those are two different savings and I am going to keep them apart, because collapsing them is the mistake this part exists to prevent.

101 101. Predict: three buffers, one message

On screen:

Part 9 · Zero-copy

A message that is three pieces in your program, and one message in the file [S14].

The program

A header, a body and a trailer — three separate buffers. You want all three in the file, in order, as one message.

Question one

How many system calls does that take — three, one, or zero?

Question two

How many times does anything copy the three buffers to make it happen?

Two answers, written separately. They are not the same question [S56].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/> [S56] [demo/scatter/README.md](https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md) — captured 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md>

Last prediction of the day, and it is two answers rather than one [S14].

Here is the situation, and you have all met it. You are writing a record to a log, or a frame to a socket. In your program it is three separate things: a header you built, a body you were given, and a trailer. In the file it has to be one message, in that order, and nobody else's line may land in the middle of it.

The obvious way is three write calls. The other obvious way is to glue the three into one buffer first and make one write call.

Question one: how many system calls. Three, one, or zero.

Question two, and this is the one I actually want. How many times does anything copy those three buffers to make it happen? Count the copies you would have to do in your own program, and the copies the kernel would have to do.

Write both numbers. I am asking separately because crossings and copies are two different costs, and almost every explanation of this topic runs them together.

Thirty seconds.

102 102. Observe: three lines become one

On screen:

Part 9 · Zero-copy

[code]

*Three crossings became one, and the return value is the whole message: 57 bytes [S56].
The three buffers never moved in the program.*

[S56] demo/scatter/README.md — `strace -e trace=write,writev`, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md>

Read the top block first. Three write calls, three crossings, and you have read lines exactly like these for two lectures now.

Now the bottom block. One call. The array inside the brackets is the argument: three entries, each with a base address and a length. That structure has a name, `iovec`, and its two fields are `iov_base` and `iov_len` — you are reading them on the screen. Three entries, and the count three after the closing bracket.

The return value is fifty-seven. Not eight. The call reports the whole message.

So question one is answered: one call, not three. The kernel gathered the three buffers for you, in order, in one crossing.

Now question two, and this is where I want you to check your own answer. How many copies did the program do to make that happen? None. The three buffers stayed exactly where they were and the kernel read them in order [S56].

But be careful with the word none, because the three-write version also copied nothing in the program. `writev` did not save you a copy. It saved you two crossings. That is the next slide.

103 103. Gather buys crossings, not copies

On screen:

Part 9 · Zero-copy

The distinction the name hides, and your second answer.

What `writev` removes

Two of the three user-to-kernel crossings, plus the staging buffer you would glue the pieces into [S58].

What it does not remove

Any copy the kernel makes further down. The four levels underneath are unchanged [S56].

`readv` is the mirror: one crossing scatters into several buffers [S58]. Valkey's reply path gathers framing, a stored string and a terminator into three `iovec` entries, then sends them with one `writev` [S65].

[S58] `writev(2)` — Linux man-pages. <https://man7.org/linux/man-pages/man2/writev.2.html> [S56] demo/scatter/README.md, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md>

[S65] `valkey/src/networking.c` — Valkey (2026). <https://github.com/valkey-io/valkey/blob/00a8a19554a39facb591837ea41ba4350260919d/src/networking.c>

This is the slide that separates the two answers, and it is the reason I asked for two numbers.

`Writev` removes crossings. Three became one. That is a real saving and it is the whole of what the call buys you.

Writev does not remove a copy. If you had written three write calls, nothing in your program would have copied either. What writev spares you is the other design: glue the header, body and trailer into one new buffer and write that. That version really does copy, and it copies for no reason except the shape of write. Removing that staging buffer is the honest way to state the win.

Below the crossing, nothing changes. The same four levels you drew this morning sit underneath a writev exactly as they sit under a write.

Readv is the same idea on the read side. One crossing, and the kernel scatters the bytes into several of your buffers instead of one [S58].

And the use I want you to remember is the boring one. Valkey's source reserves exactly three iovec entries for one reply element [S65]. They are the length prefix, the stored string itself, and the two-byte terminator. The comment in the code says why: use writev to save system calls and TCP packets [S65]. Not a headline optimisation. The removal of a buffer that only existed to satisfy the shape of an API.

104 104. Two promises, and only one of them is unconditional

On screen:

Part 9 · Zero-copy

Atomic means one thing: your bytes land as one unbroken block, with nobody else's write in the middle [S58].

- 1. **On a regular file** the promise is unconditional: "written as a single block that is not intermingled with output from writes in other processes" [S58]. A log line written with one `writev` stays whole.*
- 2. **On a pipe** it holds only below `PIPE_BUF`, 4096 bytes. Above that the kernel "may interleave the data" with another writer's [S59].*
- 3. **The trap:** the limit applies to the sum of your buffers, not the largest [inference]. Eight 1 KiB pieces are 8 KiB, and may interleave.*

I nearly put "writev is atomic" on the previous slide, and it would have been half right in the half that matters least to you. First the word, because it is used loosely. Atomic here means your bytes arrive as one unbroken block. Nobody else's write can land in the middle of yours.

On a regular file that promise is unconditional. The data goes in as a single block, not intermingled with writes from other processes [S58]. That is why a log line written with one `writew` stays whole, however many processes write to the same log.

On a pipe or a socket the rule is different, and it is the rule you met in part eight. Writes below `PIPE_BUF` must be atomic. Above it the kernel may interleave your data with another writer's [S59]. On Linux `PIPE_BUF` is four thousand and ninety-six bytes.

And the trap. The limit is on the total, not on the largest piece [inference]. Eight one-kibibyte buffers in one `writew` are eight kibibytes, twice the limit. Two processes doing that into the same pipe can interleave, and each of them checked its own biggest piece and thought it was safe.

105 105. Predict: moving a file you never look at

On screen:

Part 9 · Zero-copy

4 MiB from a file into a pipe. Two programs, same bytes [S14].

Program A

A copy loop: read 64 KiB into a buffer, write it out, repeat.

Program B

One call per chunk: `sendfile(out, in, ...)`. The program never allocates a buffer for the data.

The question

How many data system calls does each make — and how many times does each byte cross into your address space?

Neither program looks at a single byte of the file [S56].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/> [S56] [demo/scatter/README.md](https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md) — captured 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md>

Second demonstration, and the same two-part question: calls, and copies [S14].

Program A is the loop every one of you has written. Read a chunk into a buffer, write the buffer out, go round again. Four mebibytes in sixty-four kibibyte chunks.

Program B replaces the pair with one call, `sendfile`. You hand it a destination descriptor, a source descriptor and a length, and it moves the bytes. There is no buffer in your program for the data at all.

The question is the same shape as before. How many data system calls does each make. And how many times does each byte cross the boundary into your address space and back out.

The second number is the one that matters here, and it is why I am keeping the two demonstrations apart. `Writev` changed the first number. `Sendfile` changes the second.

One thing to hold on to while you predict: neither program ever looks at the bytes. Not one line in either of them inspects the data. That is the precondition, and I come back to it at the end of the part.

106 106. Observe: half the calls, and no trip through your program

On screen:

Part 9 · Zero-copy

4 MIB INTO A PIPE	DATA SYSCALLS	COPIES THROUGH USER SPACE	WALL
copy loop, 64 KiB chunks	64 read + 64 write = 128	two per chunk	2.6 ms
sendfile	64 sendfile	none	2.0 ms

The trace's tell is an absence: with `sendfile` there is **no read line at all** [S56]. The bytes went from the page cache to the pipe without visiting your address space.

Sixty-four calls because each moved 64 KiB — the pipe's capacity, not a `sendfile` limit [S56] [S59].

[S56] `demo/scatter/README.md` — `strace -c -e trace=read,write,sendfile`, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md> [S59] `pipe(7)` — Linux man-pages. <https://man7.org/linux/man-pages/man7/pipe.7.html>

Read the table across. A hundred and twenty-eight data calls against sixty-four, and two copies per chunk against none [S56].

The copies are the number I care about. In the loop, every chunk crosses into your process, sits in a buffer you never read, and crosses back out. Two copies, four mebibytes each way, to move bytes you never inspected. Sendfile hands the kernel a source, a destination and a length, and the bytes go from the page cache into the pipe [S56].

The tell in the trace is what is not there. No read lines. Part seven had the same shape on the completion side: the evidence for a mechanism that removes work is an absence, not a presence.

Now the question somebody is already asking, and it is the right question. Why sixty-four calls? The prediction offered three, one, or zero, and the answer is sixty-four. Because each call moved sixty-four kibibytes, which is what the pipe would accept at once [S56]. A pipe holds sixteen pages [S59]. That is the pipe's capacity setting the chunk size, not a limit of sendfile.

And the number I am deliberately not selling you. Two point six milliseconds against two point zero, on one machine, once. That difference would not survive a second run's noise and I am not claiming it [S56]. The claim is the copies avoided and the calls halved. Those are counted, not timed.

107 107. What the name does not cover

On screen:

Part 9 · Zero-copy

It counts the copies it removes at one boundary. No others.

1

Direct memory access stays

The bytes still move. The CPU no longer carries them [S60].

2

It needs the page cache

Level 3 is not bypassed. Handing out its pages is the mechanism [inference].

3

It buys no durability

The same four levels sit below [S67].

4

It cannot help you look

Encrypt them and the process must hold them [S60].

[S60] sendfile(2). <https://man7.org/linux/man-pages/man2/sendfile.2.html> [S67]

fsync(2). <https://man7.org/linux/man-pages/man2/fsync.2.html>

Today's thesis applied to a word rather than to a return value. That is the reason this part is in the lecture at all.

Zero-copy counts copies across one boundary: into and out of your address space. It says nothing about the others.

The two transfers at the ends remain. The disk still moves bytes into memory and the device still takes them out. What the call removed is the CPU carrying them through your program in between.

The page cache is not bypassed. It is required. Sendfile works precisely by handing the destination references to pages the kernel already has [inference]. A file with no cached pages has nothing to reference [inference]. Here is the part worth carrying. The two most famous ways to make file I/O fast, sendfile and direct I/O, cancel each other. nginx encodes exactly that in its configuration — enabling directio automatically disables sendfile for that request [S61].

It does nothing for durability. The four levels are unchanged and a sendfile that returned is a handoff like every other done today [inference].

And the precondition, which is the honest limit. Zero-copy moves bytes the kernel already has to somewhere else the kernel controls. The moment your program has to compress, encrypt or template the bytes, it must hold them, and there is nothing left to remove [S60].

108 108. Zero-copy moves bytes faster. It does not make them safer.

On screen:

Part 9 · Zero-copy

Fewer copies changes who copies, not when the bytes are safe [inference].

1. **To a file.** `sendfile` copies "within the kernel" [S60], into the page cache. Nothing is on the device until `fsync()` [S67].

2. **To a socket.** The return counts bytes the kernel accepted, not delivered: "No indication of failure to deliver is implicit in a send()" [S50].

3. **One new duty.** The kernel may still be reading your pages after the call returns: keep the file "unmodified until the reader on the other end of out_fd has consumed the transferred data" [S60].

Done still means accepted [V44].

Before you leave this part thinking zero-copy is a free upgrade, one slide on what it does not change: durability.

Zero-copy changes who does the copying, and how many times. It does not move the point at which the bytes are safe. Run it against the stack you drew this morning.

To a file. sendfile copies inside the kernel [S60], and where do the bytes land? In the page cache. Level three. Everything Part 6 told you still applies. The kernel's done is not the device's done, and nothing is on the medium until you fsync [S67].

To a socket. The return value counts the bytes the kernel accepted into its send buffer. It does not count bytes that reached the other machine. The manual for send says it flatly: no indication of failure to deliver is implicit in a send [S50].

And one thing zero-copy adds rather than removes. Because the kernel did not copy your data, it may still be reading your pages after the call has returned. The manual tells you to leave the transferred part of the file unmodified until the reader on the other end has consumed it [S60]. A copy would have freed you from that. Zero-copy does not.

So the sentence to take away is the one this whole lecture has been saying. Done means accepted by the next layer [V44]. Zero-copy makes the acceptance cheaper. It does not make it mean more.

109 109. Windows, and a silent fallback

On screen:

Part 9 · Zero-copy

Other names, and a Python fallback that says nothing.

1

Gather, a socket

WSASend with an array of WSABUF [S69].

2

Gather, a file

WriteFileGather, but every buffer must be at least an aligned page [S62].

3

Zero-copy

TransmitFile [S63].

Without `os.sendfile()`, `socket.sendfile()` uses " `send()` ... as a less efficient fallback" [S64]. No warning: on Windows you measure that [inference].

[S62] WriteFileGather — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-writefilegather> [S63] TransmitFile — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/mswsock/nf-mswsock-transmitfile> [S64] socket — Python docs. <https://docs.python.org/3/library/socket.html#socket.socket.sendfile>

No Windows capture in this part, and I say so plainly: for part nine, Windows is documentation. Parts six, seven and eight each had a traced Windows pair. This one does not, and you should weigh it accordingly.

The names first. For sockets the true counterpart of `writew` is `WSASend`. It takes an array of `WSABUF` structures, each one a pointer and a length. The page says it allows multiple send buffers, which makes it applicable to the scatter gather type of I O [S69]. For files there is `WriteFileGather`. It is not the same tool wearing another name. Every buffer must be at least a system page, and page-aligned. The handle needs the flags `FILE_FLAG_OVERLAPPED` and `FILE_FLAG_NO_BUFFERING`, so it is an overlapped handle with no cache behind it [S62]. That is a database primitive, not a general one. If you take away only one comparison, take `WSASend`.

`TransmitFile` is the zero-copy call, a file onto a socket without the round trip through your program [S63].

Now the part I most want the Windows users in the room to have, because it is a trap with your name on it. Python's `socket.sendfile` uses the real system call where it exists. Where it does not — Windows — it quietly uses `send` instead, and the documentation says so in those words: a less efficient fallback [S64]. No error. No warning. The same line of Python.

So a student who runs `sock.sendfile` on Windows, measures it, and concludes that zero-copy does nothing has measured the fallback [inference]. That is the abstraction being honest about its interface and silent about its mechanism, which is the whole subject of this course. If you do that experiment this week, check which call you actually made before you believe the number.

110 110. Homework, and the next question

On screen:

Part 10

What changed about the assignment, what you can run this week, and where Lecture 4 picks up.

Part ten. Four slides and then we are done.

Two things to settle. First, the assignment, because I told you a date in Lecture 1 and that date has moved. Second, the question Lecture 4 opens on.

And in between, the part I actually want from you this week.

111 111. Assignment 1 is not set today

On screen:

Part 10 · Close

I told you 22 September in Lecture 1. That has moved, and here is the honest reason.

What changed

The submission infrastructure is not ready. The assignments move back.

New dates

None yet. I will not put a date on a slide before I can keep it.

What it will measure

Buffered against unbuffered, and sequential against random access, over a 1 MiB file.

Predict the ratio. Measure it. Explain it in crossings.

Same method

Predict, observe, explain [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

In Lecture 1 I told you assignment one would be set on the twenty-second of September. That is today, and I am not setting it.

The reason is administrative and it is mine, not yours. The submission infrastructure is not ready. Rather than set an assignment you cannot hand in, the assignments move back.

I do not have new dates. I am not going to invent one on a slide and correct it next week. When the infrastructure is ready, you get the dates, and you get the same two-week shape I described in Lecture 1.

What I can tell you is what it will ask for, because that has not changed. Buffered against unbuffered. Sequential against random access. A one-megabyte file. You predict the ratio before you run anything, you measure it, and you explain the difference in terms of operating-system crossings [S14].

Which is exactly what we have been doing in this room for two weeks. If you have been writing predictions down, you have already rehearsed the assignment.

112 112. Voluntary homework, and three rules for it

On screen:

Part 10 · Close

Everything I ran today ships with this deck. Run it yourself.

1

Predict, then look

Write the number first. Watching alone teaches nothing [S51].

2

Report version and buffer size

Python's default binary buffer moved from 8 KiB to 128 KiB [S7].

3

Count traced, time untraced

Two runs. One for counts, one for timings [S1].

demo/durability/ · demo/waiting/ · demo/iodemo-c/ · demo/scatter/ run in the dev container. demo/destinations-win32/ needs Windows [S57].

[S51] Crouch, Fagen, Callan and Mazur (2004). <https://works.swarthmore.edu/fac-physics/203/> [S7] CPython gh-117151. <https://github.com/python/cpython/issues/117151> [S1] strace(1). <https://man7.org/linux/man-pages/man1/strace.1.html>

Here is what I want from you this week, and it is voluntary.

Everything I ran today ships with this deck. A Makefile and a Dockerfile in each folder, and a README that records what I got. Run them yourself and see whether you get the same.

Four of them run in the container on any machine. The Windows ones need a Windows machine. Those are the three I would most like somebody to check. I ran them on one virtual machine [S57]. You have faculty machines that are not it.

Three rules, and they are the same three Lecture 2 gave you.

One. Predict first, on paper, before you run anything [S51]. Do not do to yourself in your own room what I spent the first three minutes arguing against.

Two. Report your Python version and your buffer size beside every number. This is not bureaucracy. Python's default binary buffer moved from eight kilobytes to a hundred and twenty-eight [S7]. If your numbers disagree with mine, that may be the whole reason, and neither of us is wrong.

Three. Count under the tracer, time without it. Two runs [S1]. Today's waiting demonstration is why. The traced count was a floor, and the untraced rate on my laptop was two orders of magnitude higher [S53].

Bring me a number that contradicts mine. That is a good week's work.

113 113. Summary — what follows from this

On screen:

Part 10 · Close

1

A return value is a handoff

It names the layer that took them [V50].

2

*Name the boundary first**"I flushed" is ambiguous across four [V44].*

3

*Design for a wrong done**Three engines did, in a year [V40] [V56].*

4

*Crossings and copies differ**writenv removed crossings, sendfile copies [S56].**Every number today came from a run [S52] [S53] [S56]. Whose done did you measure? [V50]*

Not a recap of the part titles. This is where the four claims from the start of the hour come back. They return as four consequences, plus one rule about method.

One. From now on, read a return value as a handoff to the next layer down. It tells you which layer has taken responsibility for your bytes [V50]. It tells you nothing about the layer below that. That one habit covers most of what we looked at today [speculative].

Two. Before you reach for a fix, name the boundary. The sentence "I flushed and I still lost data" is ambiguous across four different boundaries [V44]. Say which one you crossed and the mystery usually dissolves.

Three. Design for the done being wrong. You are not going to out-think fsync-gate. Three storage engines with three different architectures reached the same answer [V41]. They replaced retry with crash and abort [V40]. You then recover from the write-ahead log [V85]. Eight years later that is still the answer [V56].

Four. When something claims to be faster, ask what it removed. Writenv removed two crossings and no copies. Sendfile removed the copies and halved the calls [S56]. Those are two different savings and a name that blurs them will cost you a wrong prediction.

And the line at the bottom is the course's method rather than today's topic. Make the boundary visible and count it [S14]. Every number I put on a slide came from a run in this room or from a capture in the demo folder [S52] [S53] [S56]. The numbers I borrowed are somebody else's measurement, and each one is cited.

Four words to carry out of the room. Whose done did you measure [V50].

114 114. What Lecture 4 answers

On screen:

Part 10 · Close

Same pattern as last week. I name it and do not answer it.

Today's window

Your bytes sat in the page cache for five seconds, in memory no process owns [S52].

In part 9

The kernel moved four mebibytes you never held — the same pages [S56].

The question

What is that memory, and whose is it?

Why it is not obvious

"Physical memory is a cache, not a container" [V72].

If we did not reach parts 7 to 9, Lecture 4 opens with them.

[S52] [demo/durability/README.md](https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md). <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md>

Last slide, and it is a question rather than an answer. Same as last week.

Today you watched your bytes sit somewhere for five seconds [S52]. I called it the page cache and you accepted it, because you had to.

And if we reached part nine, you watched the kernel move four mebibytes out of that same place into a pipe [S56]. Your program never held them. Sendfile works by handing out references to those pages. So the thing you have now met twice, from two directions, is still a name rather than an object.

So. What is that memory? Whose is it? If I ask the operating system how much free memory this machine has, does the page cache count as free or as used?

That question is harder than it sounds, and the reason is one sentence from my own note. Physical memory is a cache, not a container [V72]. So free RAM is mostly a bookkeeping category rather than a resource going to waste [inference].

Lecture 4 is that, and memory mapping with it.

And if the clock beat us today, Lecture 4 opens on the slide that said Lecture 4 starts here. We do waiting and sockets first. Nothing is lost.

Thank you. Questions, and I will stay afterwards.

115 115. References

On screen:

Part 10 · Close

- **[S1]** *strace(1)* — Linux man-pages — <https://man7.org/linux/man-pages/man1/strace.1.html>
- **[S2]** *write(2)* — Linux man-pages — <https://man7.org/linux/man-pages/man2/write.2.html>
- **[S3]** *fflush(3)* — Linux man-pages — <https://man7.org/linux/man-pages/man3/fflush.3.html>
- **[S4]** *close(2)* — Linux man-pages — <https://man7.org/linux/man-pages/man2/close.2.html>
- **[S5]** C99 draft, §7.19.3 — <https://busybox.net/~landley/c99-draft.html>
- **[S6]** Built-in Functions — *open()*, Python documentation — <https://docs.python.org/3/library/functions.html#open>
- **[S7]** Increase `io.DEFAULT_BUFFER_SIZE` — CPython issue 117151 — <https://github.com/python/cpython/issues/117151>
- **[S9]** *winternl.h* reference — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation>
- **[S10]** NtTrace — Native API tracing for Windows — <https://rogerorr.github.io/NtTrace/>
- **[S11]** End-to-End Arguments in System Design — <https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf>
- **[S13]** *fsync* — The Open Group Base Specifications — <https://pubs.opengroup.org/onlinepubs/9699919799/functions/fsync.html>
- **[S14]** Predict-Observe-Explain — <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>
- **[S15]** The sad history of the Microsoft POSIX subsystem — <https://brianreiter.org/2010/08/24/the-sad-history-of-the-microsoft-posix-subsystem/>
- **[S17]** *io.IOBase.flush()* — Python documentation — <https://docs.python.org/3/library/io.html#io.IOBase.flush>
- **[S18]** Process Monitor — Sysinternals — <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>

- **[S19]** *Inside Native Applications* — <https://learn.microsoft.com/en-us/sysinternals/resources/inside-native-applications>
- **[S20]** *os* — Miscellaneous operating system interfaces, `os.write()`, Python documentation — <https://docs.python.org/3/library/os.html#os.write>
- **[S21]** *fread(3)*, the page that documents *fwrite(3)* — Linux man-pages — <https://man7.org/linux/man-pages/man3/fwrite.3.html>
- **[S22]** *Windows API index* — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list>
- **[S31]** *ltrace(1)* — Linux man-pages — <https://man7.org/linux/man-pages/man1/ltrace.1.html>
- **[S32]** *ltrace* — project page — <https://ltrace.org/>
- **[S33]** *NtTrace* — source repository — <https://github.com/rogerorr/NtTrace>
- **[S34]** *NtTrace64 capture of cmd.exe /c echo Hello World!* — <https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/NtTrace64-CMD-ECHO-Output.txt>
- **[S36]** *NtTrace64 capture of iodemo.exe* — <https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/iodemo-c/NtTrace64-IODEMO-Output.txt>
- **[S37]** *iodemo on Linux and on Windows, the recorded runs (demo/iodemo-c/README.md)* — <https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/iodemo-c/README.md>
- **[S38]** *CloseHandle function (handleapi.h)* — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/handleapi/nf-handleapi-close-handle>
- **[S39]** *demo/close/README.md* — the double-close capture on both platforms, 2026-09-20 — <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/close/README.md>
- **[S40]** *FlushFileBuffers function (fileapi.h)* — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-flushfilebuffers>
- **[S41]** *fsyncgate* — the pgsql-hackers thread, March 2018 — <https://danluu.com/fsyncgate/>
- **[S42]** *Can Applications Recover from fsync Failures?* — USENIX ATC 2020 — <https://www.usenix.org/system/files/atc20-rebello.pdf>
- **[S43]** *PRAGMA synchronous* — SQLite documentation — https://www.sqlite.org/pragma.html#pragma_synchronous

- **[S44]** *Delayed allocation and the zero-length file problem* — Theodore Ts'o (2009) — <https://thunk.org/tytso/blog/2009/03/12/delayed-allocation-and-the-zero-length-file-problem/>
- **[S45]** *Delayed allocation and the zero-length file problem* — Valerie Aurora, LWN (2009) — <https://lwn.net/Articles/351422/>
- **[S46]** *epoll(7)* — Linux man-pages — <https://man7.org/linux/man-pages/man7/epoll.7.html>
- **[S47]** *select(2)* — Linux man-pages — <https://man7.org/linux/man-pages/man2/select.2.html>
- **[S48]** *The C10K problem* — Dan Kegel (current revision) — <http://www.kegel.com/c10k.html>
- **[S49]** *Comparing and Evaluating epoll, select, and poll Event Mechanisms* — Gammo et al. (2004) — <https://www.kernel.org/doc/ols/2004/ols2004v1-pages-215-226.pdf>
- **[S50]** *send(2)* — Linux man-pages — <https://man7.org/linux/man-pages/man2/send.2.html>
- **[S51]** *Classroom demonstrations: Learning tools or entertainment?* — Crouch, Fagen, Callan and Mazur (2004) — <https://works.swarthmore.edu/fac-physics/203/>
- **[S52]** *demo/durability/README.md* — the writeback timeline and the four-mode counts, captured 2026-09-20 — <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md>
- **[S53]** *demo/waiting/README.md* — the polling and epoll system-call counts, captured 2026-09-20 — <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/waiting/README.md>
- **[S54]** *demo/iodemo-c/README.md* — the pipe run and the traced netcat pair, captured 2026-09-20 — <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/iodemo-c/README.md>
- **[S55]** *The C10K problem* — Dan Kegel, Internet Archive snapshot of 8 May 1999 — <https://web.archive.org/web/19990508164301/http://www.kegel.com/c10k.html>
- **[S56]** *demo/scatter/README.md* — the writev trace and the sendfile counts, captured 2026-09-20 — <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md>
- **[S57]** *demo/destinations-win32/README.md* — the three-destination NtTrace capture, 2026-09-20 — <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/destinations-win32/README.md>
- **[S58]** *writev(2)* — Linux man-pages — <https://man7.org/linux/man-pages/man2/writev.2.html>

- **[S59]** `pipe(7)` — Linux man-pages — <https://man7.org/linux/man-pages/man7/pipe.7.html>
- **[S60]** `sendfile(2)` — Linux man-pages — <https://man7.org/linux/man-pages/man2/sendfile.2.html>
- **[S61]** Module `ngx_http_core_module` — nginx documentation — https://nginx.org/en/docs/http/ngx_http_core_module.html#sendfile
- **[S62]** `WriteFileGather` function (`fileapi.h`) — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-writefilegather>
- **[S63]** `TransmitFile` function (`mswsock.h`) — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/mswsock/nf-mswsock-transmitfile>
- **[S64]** `socket` — Low-level networking interface, Python documentation — <https://docs.python.org/3/library/socket.html#socket.socket.sendfile>
- **[S65]** `valkey/src/networking.c` — the reply path's `writenv` — <https://github.com/valkey-io/valkey/blob/00a8a19554a39facb591837ea41ba4350260919d/src/networking.c>
- **[S66]** `strace Wow Much Syscall` — Gregg (2014). <https://www.brendangregg.com/blog/2014-05-11/strace-wow-much-syscall.html>
- **[S67]** `fsync(2)` — Linux man-pages — <https://man7.org/linux/man-pages/man2/fsync.2.html>
- **[S68]** Linux kernel documentation — `admin-guide/sysctl/vm.rst` — <https://www.kernel.org/doc/html/latest/admin-guide/sysctl/vm.html>
- **[S69]** `WSASend` function (`winsock2.h`) — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/winsock2/nf-winsock2-wsasend>
- **[S70]** `rename(2)` — Linux man-pages — <https://man7.org/linux/man-pages/man2/rename.2.html>
- **[S71]** PostgreSQL commit `9ccdd7f` — "PANIC on `fsync()` failure" (2018) — <https://git.postgresql.org/gitweb/?p=postgresql.git;a=commit;h=9ccdd7f66e3324d2b6d3dec282cfa9ff084083f1>
- **[S72]** Named Pipes — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/ipc/named-pipes>
- **[S73]** `PeekNamedPipe` function (`namedpipeapi.h`) — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/namedpipeapi/nf-namedpipeapi-peeknamedpipe>
- **[S74]** `GetQueuedCompletionStatus` function (`ioapiset.h`) — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/ioapiset/nf-ioapiset-getqueuedcompletionstatus>

- **[S75]** *io_uring(7) — Linux man-pages — https://man7.org/linux/man-pages/man7/io_uring.7.html*
- **[V4]** *A Successful write() Is a Handoff, Not a Save — The idiom the room already writes contains no durability step.*
- **[V5]** *A Successful write() Is a Handoff, Not a Save — The limit on the thesis. The bottom of the stack is not observable from software, and the deck says so.*
- **[V6]** *Standard output is buffered differently depending on where it points — Removes the misconception that a Python write passes through the C library buffer.*
- **[V8]** *Windows and Linux place the stable syscall boundary at different layers — Takeaway 4, stated as a placement question rather than a quality judgement.*
- **[V9]** *Windows and Linux place the stable syscall boundary at different layers — Stops section 5 reading as a complaint about Windows.*
- **[V10]** *The System Call Boundary — Makes "a library call is not a system call" falsifiable with two tools instead of asserting it.*
- **[V11]** *The System Call Boundary — The closing honest-limits slide.*
- **[V14]** *Where the Abstraction Leaks — Bounds section 3's vanishing-output demonstration so it does not teach the misconception it exists to remove.*
- **[V17]** *Observing a Running System — Why sections 1 and 2 need two runs.*
- **[V20]** *File IO and Durability — The device reports success before the bytes reach the medium, which is the sentence Lecture 2 stopped on.*
- **[V21]** *NtTrace — Native API tracing for Windows (Roger Orr) — Why this tool is usable on a student's own machine, and what its design bought.*
- **[V22]** *NtTrace — Native API tracing for Windows (Roger Orr) — The single string in the Windows trace that shows the layer difference.*
- **[V23]** *NtTrace — Native API tracing for Windows (Roger Orr) — Comparison slide names the defect it has just cleared.*
- **[V24]** *PostgreSQL's fsync() surprise — Shipped storage engines answer the deferred-error problem by refusing to retry.*
- **[V25]** *Every Tracing Mechanism Trades Observation Cost for a Blind Spot — The honest-limits slide. The tool taught here is a teaching and debugging instrument, not an operational one.*
- **[V28]** *Sockets and Network I-O — One-slide bridge to networking.*
- **[V29]** *Sockets and Network I-O — The pipe row on the same slide.*
- **[V30]** *File IO and Durability — The row of the note's per-layer instrumentation table covering the kernel-to-device boundary.*
- **[V33]** *File IO and Durability — The bottom arrow of the stack diagram, and the "emptied by" cell of level 4.*

- **[V36]** *Windows and Linux place the stable syscall boundary at different layers — The caveat on the outlier list.*
- **[V37]** *File IO and Durability — The Windows half of level 3's "emptied by" cell, and of the third arrow in the stack diagram.*
- **[V38]** *Windows and Linux place the stable syscall boundary at different layers — Why Linux nominates the layer it does, and why the other platforms need not.*
- **[V39]** *Every Tracing Mechanism Trades Observation Cost for a Blind Spot — What the lower-overhead mode named in [V25] actually buys, on a named workload.*
- **[V40]** *PostgreSQL's fsync() surprise — How many storage engines abandoned the retry, and when.*
- **[V41]** *PostgreSQL's fsync() surprise — The independence of the three engines, which is what makes the never-retry slide an argument rather than an anecdote.*
- **[V43]** *Windows and Linux place the stable syscall boundary at different layers — The named list behind the outlier correction.*
- **[V44]** *File IO and Durability — Core claim, stated as a conjunction.*
- **[V45]** *File IO and Durability — The objection the room raises during section 3 — but I can read the file back.*
- **[V46]** *File IO and Durability — Windows half at equal depth.*
- **[V47]** *File IO and Durability — The honest Windows answer is undocumented, which is a stronger and truer statement than the same or safe.*
- **[V48]** *File IO and Durability — Honesty slide. Why the 40 MiB file was written back sooner than the 4 MiB one, which is the first question the observation provokes.*
- **[V49]** *File IO and Durability — Close. A shipped database declines to trust the last software handoff and defends itself another way.*
- **[V50]** *Where the Abstraction Leaks — The thesis in one sentence, from the vault's own synthesis note.*
- **[V51]** *Where the Abstraction Leaks — Fixable half. The room needs the part software can get right, or the durability section reads as fatalism.*
- **[V52]** *Where the Abstraction Leaks — Counter-position. Trusting the abstraction is respectable, and the note says exactly where it stops being respectable.*
- **[V53]** *A failed fsync() is not retryable — The kernel's done observed lying, in a trace of exactly the kind this course teaches the room to read.*
- **[V54]** *A failed fsync() is not retryable — Why the obvious repair is not a repair. It has the same shape as Lecture 2's slide "Never retry a failed close", one level down.*
- **[V55]** *A failed fsync() is not retryable — Windows slide. The asymmetry is about documentation, not about which platform is safer.*

- **[V56]** PostgreSQL's fsync() surprise — The industry answer to an untrustworthy done is crash and replay, and it was never meant to last this long.
- **[V57]** Readiness and Completion Models — Buffer ownership, not API taste, is what separates the two models, and it is why porting between them is a rewrite.
- **[V58]** Readiness and Completion Models — The sentence that stops the room reading ready as a guarantee, which is the misconception this model produces.
- **[V59]** Readiness and Completion Models — Correction to its own framing.
- **[V60]** Readiness and Completion Models — Windows slide. What the room should carry away as present tense, and what is only direction of travel.
- **[V61]** The C10K Problem Was About the Cost of Waiting, Not I-O — Predict slide. The controlled experiment behind the question, stated as what was held fixed and what was varied.
- **[V62]** The C10K Problem Was About the Cost of Waiting, Not I-O — Explanation. `epoll` did not make a read faster, and the room needs that said before it draws the wrong conclusion from the counts.
- **[V63]** The C10K Problem Was About the Cost of Waiting, Not I-O — The page most often cited for this argument did not make it in 1999, so the deck cites the revision and the measurement instead.
- **[V64]** Sockets and Network I-O — Mechanism. Why `read` and `write` work on a socket, stated as a kernel structure rather than as a convention.
- **[V65]** Sockets and Network I-O — Boundary. The same interface does not mean the same contract, and the note names where the default stops.
- **[V66]** Sockets and Network I-O — Windows slide. The platforms agree on the mechanism and disagree on which layer they promise, which is the accurate statement.
- **[V67]** Interprocess Communication — Boundary. The clean one-write-one-read result in the capture holds because fourteen bytes is small, and the room should know the threshold.
- **[V68]** `io_uring` and Windows IoRing — Completion on Both Platforms — Close. Where both platforms ended up, stated precisely enough not to read as "they converged".
- **[V69]** Blocking Is a Scheduler Decision, Not a Property of the I-O — The one question that orders polling, readiness and completion into a ladder rather than a feature list.
- **[V70]** Network Buffering and Delivery Semantics — Boundary. The capture shows one write and one read only because the payload is fourteen bytes, and the slide must say so.

- **[V71]** *Standard output is buffered differently depending on where it points — Honest bound on its own demonstration.*
- **[V72]** *Virtual Memory and the Page Cache — Handover. The page cache held the bytes for five seconds, and what that memory actually is belongs to the next lecture.*
- **[V73]** *io_uring and Windows IoRing — Completion on Both Platforms — Boundary. The demonstration uses pipes, and a student who tries the same loop on a file gets a mechanism that always answers yes.*
- **[V74]** *Readiness and Completion Models — What a completion model demands of the program, in Microsoft's own words, and the memory bill that follows from it.*
- **[V75]** *Readiness and Completion Models — The other half of the ownership comparison.*
- **[V76]** *Readiness and Completion Models — Why the choice of model is structural rather than a matter of API taste.*
- **[V77]** *Readiness and Completion Models — Takeaway 2 and section 4's definition slide.*
- **[V78]** *File IO and Durability — What a dirty page is, what writeback is, and that the kernel rather than the program schedules it.*
- **[V79]** *io_uring and Windows IoRing — Completion on Both Platforms — Close. The version in which Linux acquired a completion interface.*
- **[V80]** *io_uring and Windows IoRing — Completion on Both Platforms — Close. The build in which Windows acquired a ring, paired with [V79].*
- **[V81]** *Readiness and Completion Models — Correction. The control case that shows readiness is the Unix family's answer rather than a Linux quirk.*
- **[V82]** *Interprocess Communication — Boundary. The threshold in bytes, and what happens above it, which the atomicity span [V67] does not state.*
- **[V83]** *A failed fsync() is not retryable — Windows slide. The mechanism behind the undocumented contract, which is why no single answer exists.*
- **[V84]** *PostgreSQL's fsync() surprise — What InnoDB actually changed, which the count-and-year span [V40] does not carry.*
- **[V85]** *A failed fsync() is not retryable — Close. The log-and-replay answer, in PostgreSQL's own shipped documentation rather than as the deck's summary of it.*
- **[V86]** *File IO and Durability — The device lying is a reported practice rather than a theoretical possibility, and a shipped database says so.*
- **[V87]** *Sockets and Network I-O — Boundary. A concrete operation that does not transfer, which the strong-default span [V65] asserts without instanting.*

- **[V88]** *io_uring and Windows IoRing — Completion on Both Platforms — Why the regular-file gap mattered enough to build a second model, which the `epoll`-and-regular-files span [V73] does not say.*
- **[V89]** *A failed `fsync()` is not retryable — The mechanism behind the failed retry, which [V54]'s one-sentence span states without explaining.*
- **[V90]** *A failed `fsync()` is not retryable — How long the misreading stood before anyone noticed, which is what makes it a contract failure rather than a careless bug.*
- **[V91]** *File IO and Durability — Windows slide. The judgement itself, from the note that makes it, rather than asserted by the deck on the strength of a missing wiki row.*
- **[V92]** *Readiness and Completion Models — The `select(2)` BUGS caveat applies to the interface the demonstration actually used.*
- **[V93]** *PostgreSQL's `fsync()` surprise — What PostgreSQL actually changed, and that the change reached every release rather than only the current one.*
- **[V94]** *The C10K Problem Was About the Cost of Waiting, Not I-O — Kegel's arithmetic for why a thread per connection ran out of virtual memory.*
- **[V95]** *The C10K Problem Was About the Cost of Waiting, Not I-O — Banga, Mogul and Druschel on the scheduler half of the same bill.*
- **[V96]** *The C10K Problem Was About the Cost of Waiting, Not I-O — What unifies the thread rung with the polling rung: a charge levied per connection, not per byte.*

The handout PDF carries the full reference list, and a one-page summary of every [V] source with its own external references.

Every source behind every claim today, in the order the deck used them.

The S entries are external and every one has a link you can follow tonight. Most of them are manual pages, vendor documentation and standards. They are short, they are free, and they are written by the people who define the behaviour.

Several of the S entries are our own captures — the Windows traces and the container runs. Those ship with the deck, in the demo folder, so you can check my numbers against the files rather than against my slides.

The V entries are notes from my own research vault. You cannot open those, so the handout carries a one-page summary of each one together with every external source it draws on.